

The Ultimate Simulation Cheat Sheet

Parsa

2026-09-21

Contents

1	How to Use This Cheat Sheet	4
2	Part 0 — The 30-Second Strategy	5
3	Part 1 — Foundations	6
3.1	1.1 What is simulation?	6
3.2	1.2 The data-generating process (DGP)	6
3.3	1.3 Reproducibility	6
3.4	1.4 Why randomness needs many runs	6
3.5	1.5 The Monte Carlo error	6
3.6	1.6 Notation	7
3.7	1.7 The three quantities you simulate	7
4	Part 2 — Random Number Generation	7
4.1	2.1 Seeding	7
4.2	2.2 Basic generators	7
4.3	2.3 Reproducibility across functions	8
4.4	2.4 Sampling	8
4.5	2.5 Sampling with strata (stratified)	8
4.6	2.6 Shuffling and permutation	9
4.7	2.7 Random splits for cross-validation	9
4.8	2.8 Reproducible random sampling for train/test	9
5	Part 3 — Distributions Reference	9
5.1	3.1 Normal	10
5.2	3.2 Uniform	10
5.3	3.3 Binomial	10
5.4	3.4 Poisson	11
5.5	3.5 Exponential	11
5.6	3.6 Chi-square, t, F	11
5.7	3.7 Multivariate normal	12
5.8	3.8 Truncated distributions	12
5.9	3.9 Discrete distributions	12
5.10	3.10 Verifying a generator	12
6	Part 4 — Monte Carlo Estimation	13
6.1	4.1 Estimating a mean	13
6.2	4.2 Estimating a variance	13
6.3	4.3 Estimating a probability	13
6.4	4.4 Estimating a quantile	14
6.5	4.5 Estimating a nonlinear function	14

6.6	4.6 Estimating a tail probability	14
6.7	4.7 Estimating a joint probability	14
6.8	4.8 Monte Carlo summary table	14
6.9	4.9 Report template	15
7	Part 5 — Bias, Variance, MSE	15
7.1	5.1 Setup	15
7.2	5.2 Bias of the sample mean (should be 0)	15
7.3	5.3 Bias of the sample median	15
7.4	5.4 Bias of the sample variance (using n)	16
7.5	5.5 Bias of the ratio estimator	16
7.6	5.6 Comparing two estimators	17
7.7	5.7 Relative efficiency	17
7.8	5.8 Report template	17
8	Part 6 — Confidence Intervals & Coverage	18
8.1	6.1 Coverage of a CI	18
8.2	6.2 Coverage with a bad CI (wrong SE)	18
8.3	6.3 Coverage with FPC (finite population)	18
8.4	6.4 Coverage of a bootstrap CI	19
8.5	6.5 CI width as a function of n	19
8.6	6.6 Report template	20
9	Part 7 — Bootstrap	20
9.1	7.1 The idea	20
9.2	7.2 Bootstrap SE of a mean	20
9.3	7.3 Bootstrap CI: percentile method	20
9.4	7.4 Bootstrap CI: basic (reverse percentile)	21
9.5	7.5 Bootstrap CI: BCa (bias-corrected accelerated)	21
9.6	7.6 Bootstrap for a correlation	21
9.7	7.7 Bootstrap for a regression slope	21
9.8	7.8 Bootstrap for a proportion	22
9.9	7.9 Block bootstrap for time series	22
9.10	7.10 Parametric bootstrap	22
9.11	7.11 Cluster bootstrap	23
9.12	7.12 Bootstrap summary	23
9.13	7.13 Report template	23
10	Part 8 — Permutation Tests	23
10.1	8.1 The idea	23
10.2	8.2 Two-sample permutation test	24
10.3	8.3 Permutation test for correlation	24
10.4	8.4 Permutation test for a regression slope	24
10.5	8.5 Paired permutation test	25
10.6	8.6 Permutation test for ANOVA	25
10.7	8.7 Report template	25
11	Part 9 — Power Simulation	25
11.1	9.1 What is power?	25
11.2	9.2 Power of a two-sample t-test	25
11.3	9.3 Power as a function of effect size	26
11.4	9.4 Power as a function of n	26
11.5	9.5 Power of a chi-square test	27
11.6	9.6 Power of ANOVA	27
11.7	9.7 Power of a regression test	28

11.8	9.8	Power curve plot	28
11.9	9.9	Report template	29
12	Part 10	— Sample Size via Simulation	29
12.1	10.1	Find n for target power	29
12.2	10.2	Binary search for n	30
12.3	10.3	Sample size for a proportion test	30
12.4	10.4	Sample size for a chi-square	30
12.5	10.5	Report template	31
13	Part 11	— Simulating Specific Designs	31
13.1	11.1	Simulating a regression dataset	31
13.2	11.2	Simulating a logistic regression dataset	31
13.3	11.3	Simulating a Poisson regression dataset	32
13.4	11.4	Simulating a one-way ANOVA	32
13.5	11.5	Simulating a two-way ANOVA	32
13.6	11.6	Simulating repeated measures	32
13.7	11.7	Simulating an AR(1) time series	33
13.8	11.8	Simulating a random walk	33
13.9	11.9	Simulating an ARIMA(p,d,q)	34
13.10	11.10	Simulating a seasonal series	35
13.11	11.11	Simulating stratified data	36
13.12	11.12	Simulating clustered data	36
13.13	11.13	Simulating a contingency table	37
13.14	11.14	Simulating multivariate normal	37
14	Part 12	— Custom Distributions	37
14.1	12.1	Inverse-transform sampling	37
14.2	12.2	Simulating an exponential from uniform	38
14.3	12.3	Rejection sampling	38
14.4	12.4	Inverse CDF for a discrete distribution	39
14.5	12.5	Sampling from a mixture	39
14.6	12.6	Report template	40
15	Part 13	— Monte Carlo Integration	40
15.1	13.1	Basic idea	40
15.2	13.2	Example: $\int_0^1 e^{-x^2} dx$	40
15.3	13.3	Example: $\int_0^\pi \sin(x) dx = 2$	41
15.4	13.4	Example: $\int_{-\infty}^\infty \phi(x) dx = 1$ (check normal)	41
15.5	13.5	Multidimensional integration	41
15.6	13.6	MC integration with importance sampling	41
15.7	13.7	Example: rare event probability via importance sampling	42
15.8	13.8	Report template	42
16	Part 14	— Markov Chains & Random Walks	42
16.1	14.1	Simple random walk on integers	42
16.2	14.2	Brownian motion (continuous random walk)	43
16.3	14.3	Markov chain simulation	44
16.4	14.4	Metropolis-Hastings (basic)	45
16.5	14.5	Random walk on a graph	46
16.6	14.6	Report template	46
17	Part 15	— Variance Reduction	46
17.1	15.1	Antithetic variables	46

17.2	15.2	Control variates	47
17.3	15.3	Importance sampling (already in Part 13)	47
17.4	15.4	Stratified sampling for simulation	47
17.5	15.5	Common random numbers (for comparisons)	47
17.6	15.6	Report template	47
18	Part 16	— Jackknife & Cross-Validation	48
18.1	16.1	Jackknife	48
18.2	16.2	LOOCV for regression	48
18.3	16.3	K-fold cross-validation	48
18.4	16.4	Cross-validation for model selection	49
18.5	16.5	Report template	49
19	Part 17	— Simulating Hypothesis Tests Under H_0	50
19.1	17.1	Simulate H_0 to get the null distribution	50
19.2	17.2	Simulate type I error for a chi-square test	50
19.3	17.3	Under the alternative, simulate power	51
19.4	17.4	Simulating p-value distributions under H_1	51
19.5	17.5	Report template	51
20	Part 18	— Efficiency in R	51
20.1	18.1	Preallocate vectors	51
20.2	18.2	Use <code>replicate</code> over explicit loops	51
20.3	18.3	Vectorize when possible	52
20.4	18.4	Benchmarking	52
20.5	18.5	Parallelisation	52
21	Part 19	— Theory Q&A	52
22	Part 20	— Exam Scenarios	54
22.1	20.1	“Estimate the mean of X by simulation”	54
22.2	20.2	“Estimate the bias and MSE of an estimator”	54
22.3	20.3	“Compute coverage of a 95% CI”	54
22.4	20.4	“Find the sample size for 80% power”	54
22.5	20.5	“Bootstrap the standard error of a statistic”	54
22.6	20.6	“Bootstrap a CI”	54
22.7	20.7	“Permutation test for group difference”	54
22.8	20.8	“Simulate from a custom distribution”	55
22.9	20.9	“Simulate data for a given design”	55
22.10	20.10	“Monte Carlo integration”	55
23	Part 21	— Quick Reference	55
24	Part 22	— Common Mistakes	56
25	Part 23	— The Single Most Important Idea	57
25.1		What is covered	57

1 How to Use This Cheat Sheet

Topic	Section
30-second strategy	0
Foundations: what and why	1
Random number generation	2

Topic	Section
Distributions reference	3
Monte Carlo estimation	4
Bias, variance, MSE	5
Confidence intervals & coverage	6
Bootstrap	7
Permutation tests	8
Power simulation	9
Sample size via simulation	10
Simulating specific designs	11
Custom distributions (inverse, rejection)	12
Monte Carlo integration	13
Markov chains & random walks	14
Variance reduction	15
Jackknife & cross-validation	16
Simulating hypothesis tests under H_0	17
Efficiency in R	18
Theory Q&A	19
Exam scenarios	20
Quick reference	21
Common mistakes	22
Single most important idea	23

Golden rule: every chunk is self-contained.

2 Part 0 — The 30-Second Strategy

For **any** simulation question, run this pipeline:

1. **What is being estimated or tested?** A mean, a variance, a bias, a p-value, a coverage, a power?
2. **What is the data-generating process (DGP)?** Write it down in code once.
3. **Choose B , the number of replications.** Rule of thumb: $B \geq 1000$ for estimation, $B \geq 5000$ for p-values/coverage.
4. **Loop or replicate:** generate data, compute the statistic, store the result.
5. **Summarise:** mean, sd, quantile.
6. **Report:** the estimated quantity, its Monte Carlo SE, and an interpretation.

```
set.seed(1)
B <- 5000
results <- replicate(B, {
  # 1. Generate data
  # 2. Compute statistic
  # 3. Return it
})
mean(results); sd(results)
```

If you can do these six steps, you can answer any simulation question.

3 Part 1 — Foundations

3.1 1.1 What is simulation?

Simulation (Monte Carlo) means running a **random experiment** many times to estimate a quantity that is hard to compute analytically.

Three uses: 1. **Estimate** quantities with no closed form (MSE of median, coverage of a complex CI). 2. **Validate** theory (does my CI achieve 95% coverage?). 3. **Explore** designs before collecting data (power, sample size).

3.2 1.2 The data-generating process (DGP)

Every simulation starts with a **recipe** for producing data:

- **Distribution:** $y \sim N(\mu, \sigma^2)$.
- **Sample size:** n .
- **Design:** one group, multiple groups, paired, time series.

Write the DGP in code **once**, then wrap it in `replicate()`.

3.3 1.3 Reproducibility

Always call `set.seed()` before generating random numbers:

```
# --- Self-contained: reproducibility ---
set.seed(101)
p1_a <- rnorm(3)

set.seed(101)
p1_b <- rnorm(3)

identical(p1_a, p1_b)
```

```
## [1] TRUE
```

Rule: Choose a seed (any integer), record it, and don't change it between runs.

3.4 1.4 Why randomness needs many runs

A single random sample gives one noisy estimate. The **law of large numbers** says the average over B runs converges to the truth.

```
# --- Self-contained: convergence ---
set.seed(102)
p1_means <- cumsum(rnorm(5000)) / 1:5000 # running mean of N(0,1)
tail(p1_means, 5)
```

```
## [1] 0.01190051 0.01181174 0.01146600 0.01140292 0.01137720
```

Interpretation: The running mean settles near 0 as B grows.

3.5 1.5 The Monte Carlo error

The standard error of a Monte Carlo estimate of a mean is:

$$\text{MC-SE} = \frac{s}{\sqrt{B}}$$

where s = sample SD of the B simulated values.

Rule of thumb: To halve the MC-SE, quadruple B .

3.6 1.6 Notation

Symbol	Meaning
B	Number of replications
$\hat{\theta}_b$	Estimate from replication b
$\hat{\theta}$	True estimand (usually unknown)
$\bar{\hat{\theta}}$	Average over replications
s	SD of the B estimates
MC-SE	Monte Carlo standard error = $s/\sqrt{B}$

3.7 1.7 The three quantities you simulate

1. **Bias:** $\widehat{\text{Bias}} = \bar{\hat{\theta}} - \theta$.
2. **Variance:** $\widehat{\text{Var}} = s^2$.
3. **MSE:** $\widehat{\text{MSE}} = \widehat{\text{Bias}}^2 + \widehat{\text{Var}}$.

Where θ is the true parameter you chose when setting up the DGP.

4 Part 2 — Random Number Generation

4.1 2.1 Seeding

```
# --- Self-contained: set.seed ---
set.seed(201)
p2_x <- rnorm(5)
p2_x
```

```
## [1] 0.2860607 -0.3445460 0.3257637 -1.6965810 -1.2854825
```

Rules: - One seed per simulation study. - If you change the seed, the answer changes. - Never reseed inside the loop.

4.2 2.2 Basic generators

Function	Distribution
<code>rnorm(n, mean, sd)</code>	Normal
<code>runif(n, min, max)</code>	Uniform
<code>rbinom(n, size, prob)</code>	Binomial
<code>rpois(n, lambda)</code>	Poisson
<code>rexp(n, rate)</code>	Exponential
<code>rgamma(n, shape, rate)</code>	Gamma
<code>rbeta(n, a, b)</code>	Beta
<code>rchisq(n, df)</code>	Chi-square
<code>rt(n, df)</code>	t
<code>rf(n, df1, df2)</code>	F
<code>rmultinom(n, size, prob)</code>	Multinomial
<code>sample(x, n, replace)</code>	From a vector

4.3 2.3 Reproducibility across functions

```
# --- Self-contained: same seed, different generators ---
set.seed(202)
p2_a <- rnorm(3)

set.seed(202)
p2_b <- rnorm(3)
p2_c <- runif(3)    # advances RNG state

identical(p2_a, p2_b)
```

```
## [1] TRUE
```

Note: Different generators consume the RNG stream differently. Use one seed per experiment.

4.4 2.4 Sampling

```
# --- Self-contained: sampling options ---
set.seed(203)
p2_pop <- 1:10

sample(p2_pop, 5)                                # without replacement

## [1]  6  5 10  1  3

sample(p2_pop, 5, replace = TRUE)                # with replacement

## [1]  2  4  4  6 10

sample(p2_pop, 5, prob = 1:10)                   # weighted

## [1]  9  6  4 10  8

sample(c("H", "T"), 10, replace = TRUE,
       prob = c(0.5, 0.5))                       # coin flips

## [1] "T" "T" "T" "H" "H" "T" "T" "T" "H" "T"
```

4.5 2.5 Sampling with strata (stratified)

```
# --- Self-contained: stratified sampling ---
set.seed(204)
p2_strata <- c(rep("A", 100), rep("B", 200), rep("C", 300))
p2_values <- rnorm(600, 50, 10)

# Proportional allocation: n = 60
p2_n <- 60
p2_N <- table(p2_strata)
p2_nh <- round(p2_n * p2_N / sum(p2_N))

p2_sample <- unlist(lapply(names(p2_N), function(h) {
  idx <- which(p2_strata == h)
  sample(idx, p2_nh[h])
}))
length(p2_sample); table(p2_strata[p2_sample])
```

```
## [1] 60
##
## A B C
## 10 20 30
```

4.6 2.6 Shuffling and permutation

```
# --- Self-contained: shuffling ---
set.seed(205)
p2_x <- 1:10
sample(p2_x)           # permutation

## [1] 6 4 1 3 8 5 7 2 10 9

sample(p2_x)[1:3]      # random subset

## [1] 6 3 8
```

4.7 2.7 Random splits for cross-validation

```
# --- Self-contained: k-fold indices ---
set.seed(206)
p2_n <- 100
p2_K <- 5
p2_folds <- sample(rep(1:p2_K, length.out = p2_n))
table(p2_folds)

## p2_folds
## 1 2 3 4 5
## 20 20 20 20 20
```

4.8 2.8 Reproducible random sampling for train/test

```
# --- Self-contained: train/test split ---
set.seed(207)
p2_df <- tibble(id = 1:100, y = rnorm(100))
p2_idx <- sample(1:nrow(p2_df), size = 0.8 * nrow(p2_df))
p2_train <- p2_df[p2_idx, ]
p2_test  <- p2_df[-p2_idx, ]
nrow(p2_train); nrow(p2_test)

## [1] 80
## [1] 20
```

5 Part 3 — Distributions Reference

For every distribution R provides four functions:

Prefix	Function	Example
d	Density / PMF	<code>dnorm(0)</code>
p	CDF	<code>pnorm(1.96)</code>

Prefix	Function	Example
q	Quantile (inverse CDF)	qnorm(0.975)
r	Random draw	rnorm(5)

5.1 3.1 Normal

```
# --- Self-contained: normal ---
dnorm(0)           # density at 0

## [1] 0.3989423

pnorm(1.96)        # P(X <= 1.96)

## [1] 0.9750021

qnorm(0.975)       # 97.5% quantile = 1.96

## [1] 1.959964

set.seed(301); rnorm(3)

## [1] 0.2458642 -2.8571924 0.2901439
```

5.2 3.2 Uniform

```
# --- Self-contained: uniform ---
dunif(0.5)         # density at 0.5

## [1] 1

punif(0.5)         # P(X <= 0.5)

## [1] 0.5

qunif(0.5)         # median

## [1] 0.5

set.seed(302); runif(3)

## [1] 0.5893353 0.8965255 0.6649432
```

5.3 3.3 Binomial

```
# --- Self-contained: binomial ---
dbinom(3, size = 10, prob = 0.5) # P(X = 3)

## [1] 0.1171875

pbinom(3, size = 10, prob = 0.5) # P(X <= 3)

## [1] 0.171875

qbinom(0.95, size = 10, prob = 0.5)

## [1] 8

set.seed(303); rbinom(5, size = 10, prob = 0.5)
```

```
## [1] 4 7 5 6 6
```

5.4 3.4 Poisson

```
# --- Self-contained: Poisson ---  
dpois(2, lambda = 3)          #  $P(X = 2)$ 
```

```
## [1] 0.2240418
```

```
ppois(2, lambda = 3)          #  $P(X \leq 2)$ 
```

```
## [1] 0.4231901
```

```
qpois(0.95, lambda = 3)
```

```
## [1] 6
```

```
set.seed(304); rpois(5, lambda = 3)
```

```
## [1] 7 4 0 2 8
```

5.5 3.5 Exponential

```
# --- Self-contained: exponential ---  
dexp(1, rate = 2)
```

```
## [1] 0.2706706
```

```
pexp(1, rate = 2)
```

```
## [1] 0.8646647
```

```
qexp(0.5, rate = 2)
```

```
## [1] 0.3465736
```

```
set.seed(305); rexp(5, rate = 2)
```

```
## [1] 0.30041185 0.09147062 0.07123932 0.24336937 0.15075283
```

5.6 3.6 Chi-square, t, F

```
# --- Self-contained: chisq, t, F ---  
pchisq(3.84, df = 1)          # 0.95
```

```
## [1] 0.9499565
```

```
qt(0.975, df = 10)           # 97.5% t quantile
```

```
## [1] 2.228139
```

```
qf(0.95, df1 = 2, df2 = 10)   # 95% F quantile
```

```
## [1] 4.102821
```

```
set.seed(306); rchisq(3, df = 5); rt(3, df = 10); rf(3, df1 = 2, df2 = 5)
```

```
## [1] 3.3230184 6.1178245 0.9654254
```

```
## [1] 0.30805341 -1.05001470 0.02027287
```

```
## [1] 0.8622922 0.1835217 0.7775511
```

5.7 3.7 Multivariate normal

```
# --- Self-contained: MVN ---
set.seed(307)
p3_mu    <- c(0, 0)
p3_Sigma <- matrix(c(1, 0.7, 0.7, 1), nrow = 2)

# Manual MVN draw
p3_z <- matrix(rnorm(10 * 2), ncol = 2)
p3_x <- p3_z %*% chol(p3_Sigma) + matrix(p3_mu, nrow = 10, ncol = 2, byrow = TRUE)
round(head(p3_x, 3), 3)
```

```
##           [,1] [,2]
## [1,]  0.578 0.261
## [2,] -0.503 -0.388
## [3,] -1.825 -1.226
```

Why `chol`: The Cholesky factor of Σ gives the “square root” of the covariance matrix.

5.8 3.8 Truncated distributions

```
# --- Self-contained: truncated normal (rejection) ---
set.seed(308)
p3_n <- 1000
p3_x <- rnorm(p3_n)
p3_trunc <- p3_x[p3_x > 0]      # keep only values > 0
length(p3_trunc)
```

```
## [1] 496
```

5.9 3.9 Discrete distributions

```
# --- Self-contained: general discrete ---
set.seed(309)
p3_vals  <- c(0, 1, 2, 5)
p3_probs <- c(0.2, 0.4, 0.3, 0.1)

p3_draws <- sample(p3_vals, 1000, replace = TRUE, prob = p3_probs)
table(p3_draws) / 1000
```

```
## p3_draws
##      0      1      2      5
## 0.185 0.415 0.298 0.102
```

5.10 3.10 Verifying a generator

```
# --- Self-contained: verify generator ---
set.seed(310)
p3_x <- rnorm(10000, mean = 5, sd = 2)
c(mean = mean(p3_x), sd = sd(p3_x))
```

```
##      mean      sd
```

```
## 5.013247 2.012622
```

Interpretation: The empirical mean and SD should be close to the theoretical values (5 and 2).

6 Part 4 — Monte Carlo Estimation

6.1 4.1 Estimating a mean

Goal: Estimate $E[X]$ for $X \sim N(50, 100)$.

```
# --- Self-contained: MC estimate of a mean ---  
set.seed(401)  
p4_B <- 5000  
p4_x <- rnorm(p4_B, mean = 50, sd = 10)  
mean(p4_x)
```

```
## [1] 49.94548
```

```
sd(p4_x) / sqrt(p4_B)
```

```
## [1] 0.1416857
```

Interpretation: Estimate = mean of the draws. MC-SE = $\text{sd}/\sqrt{B}$. The theoretical answer is 50.

6.2 4.2 Estimating a variance

```
# --- Self-contained: MC estimate of variance ---  
set.seed(402)  
p4_B <- 5000  
p4_x <- rnorm(p4_B, mean = 50, sd = 10)  
var(p4_x)
```

```
## [1] 96.03619
```

Theoretical: 100.

6.3 4.3 Estimating a probability

Goal: $P(X > 2)$ where $X \sim N(0, 1)$.

```
# --- Self-contained: MC estimate of a probability ---  
set.seed(403)  
p4_B <- 10000  
p4_x <- rnorm(p4_B)  
mean(p4_x > 2) # estimated probability
```

```
## [1] 0.0238
```

```
sqrt(mean(p4_x > 2) * (1 - mean(p4_x > 2)) / p4_B) # MC-SE
```

```
## [1] 0.001524256
```

Theoretical: $1 - \Phi(2) = 0.0227501$.

6.4 4.4 Estimating a quantile

```
# --- Self-contained: MC estimate of a quantile ---
set.seed(404)
p4_B <- 10000
p4_x <- rnorm(p4_B)
quantile(p4_x, 0.975)
```

```
##      97.5%
```

```
## 1.925028
```

Theoretical: $qnorm(0.975) = 1.96$.

6.5 4.5 Estimating a nonlinear function

Goal: $E[X^2]$ for $X \sim U(0, 1)$.

```
# --- Self-contained: MC estimate of a nonlinear function ---
set.seed(405)
p4_B <- 5000
p4_x <- runif(p4_B)
mean(p4_x^2)
```

```
## [1] 0.3312185
```

Theoretical: $E[X^2] = 1/3 \approx 0.333$.

6.6 4.6 Estimating a tail probability

```
# --- Self-contained: rare event ---
set.seed(406)
p4_B <- 100000
p4_x <- rnorm(p4_B)
mean(p4_x > 4)           # P(X > 4)  3.17e-5
```

```
## [1] 1e-05
```

Warning: For very rare events, use importance sampling (Part 15) to reduce variance.

6.7 4.7 Estimating a joint probability

Goal: $P(X > 0, Y > 0)$ where (X, Y) are jointly normal with $\rho = 0.5$.

```
# --- Self-contained: joint probability ---
set.seed(407)
p4_B <- 10000
p4_Sigma <- matrix(c(1, 0.5, 0.5, 1), nrow = 2)
p4_z <- matrix(rnorm(p4_B * 2), ncol = 2)
p4_x <- p4_z %*% chol(p4_Sigma)
mean(p4_x[,1] > 0 & p4_x[,2] > 0)
```

```
## [1] 0.3389
```

Interpretation: Estimate of the joint probability.

6.8 4.8 Monte Carlo summary table

Quantity	Estimator	MC-SE
Mean	<code>mean(x)</code>	<code>sd(x)/sqrt(B)</code>
Variance	<code>var(x)</code>	<code>sd(x^2)/sqrt(B) - Var (approx)</code>
Probability	<code>mean(x > c)</code>	<code>sqrt(p(1-p)/B)</code>
Quantile	<code>quantile(x, p)</code>	via bootstrap
$\int f(x)dx$	<code>mean(f(x))</code>	<code>sd(f(x))/sqrt(B)</code>

6.9 4.9 Report template

“Monte Carlo with $B = 10000$ replications estimates the mean at $\hat{\theta} = 49.98$ with Monte Carlo SE 0.10. The theoretical value is 50.”

7 Part 5 — Bias, Variance, MSE

7.1 5.1 Setup

Choose a **true parameter** θ . Estimate it B times. Compute bias, variance, MSE.

$$\text{Bias} = \bar{\hat{\theta}} - \theta, \quad \text{Var} = s^2, \quad \text{MSE} = \text{Bias}^2 + \text{Var}$$

7.2 5.2 Bias of the sample mean (should be 0)

```
# --- Self-contained: bias of the sample mean ---
set.seed(501)
p5_B <- 5000
p5_n <- 30
p5_mu <- 10

p5_means <- replicate(p5_B, mean(rnorm(p5_n, mean = p5_mu)))

c(bias = mean(p5_means) - p5_mu,
  var = var(p5_means),
  mse = mean((p5_means - p5_mu)^2))
```

```
##          bias          var          mse
## 0.002111369 0.032146592 0.032144621
```

Theoretical: Bias = 0, Var = $\sigma^2/n = 1/30 \approx 0.033$, MSE = 0.033.

7.3 5.3 Bias of the sample median

```
# --- Self-contained: bias of the median ---
set.seed(502)
p5_B <- 5000
p5_n <- 30
p5_mu <- 0

p5_med <- replicate(p5_B, median(rnorm(p5_n, mean = p5_mu)))

c(bias = mean(p5_med) - p5_mu,
```

```
var = var(p5_med),
mse = mean((p5_med - p5_mu)^2))
```

```
##          bias          var          mse
## -0.0005929135  0.0510285794  0.0510187252
```

Interpretation: Median may be slightly biased for small n ; variance is larger than mean's variance.

7.4 5.4 Bias of the sample variance (using n)

If you use $\frac{1}{n} \sum (y_i - \bar{y})^2$, the estimator is biased.

```
# --- Self-contained: biased variance estimator ---
set.seed(503)
p5_B <- 5000
p5_n <- 20
p5_sigma2 <- 4

p5_biased <- replicate(p5_B, {
  y <- rnorm(p5_n, sd = sqrt(p5_sigma2))
  mean((y - mean(y))^2)
})

c(bias = mean(p5_biased) - p5_sigma2)
```

```
##          bias
## -0.1916275
```

Theoretical: Bias = $-\sigma^2/n = -0.2$.

7.5 5.5 Bias of the ratio estimator

```
# --- Self-contained: bias of ratio estimator ---
set.seed(504)
p5_B <- 5000
p5_n <- 30
p5_N <- 500
p5_x <- rnorm(p5_N, mean = 20, sd = 4)
p5_y <- 3 * p5_x + rnorm(p5_N, sd = 3)
p5_mux <- mean(p5_x)
p5_muy <- mean(p5_y)

p5_ratio <- replicate(p5_B, {
  idx <- sample(p5_N, p5_n)
  mean(p5_y[idx]) / mean(p5_x[idx]) * p5_mux
})

c(true = p5_muy,
  bias = mean(p5_ratio) - p5_muy,
  var = var(p5_ratio),
  mse = mean((p5_ratio - p5_muy)^2))
```

```
##          true          bias          var          mse
## 60.473840230  0.005351776  0.268339302  0.268314276
```

Interpretation: Ratio estimator is biased but often has smaller MSE than the sample mean.

7.6 5.6 Comparing two estimators

```
# --- Self-contained: mean vs median ---
set.seed(505)
p5_B <- 5000
p5_n <- 30
p5_mu <- 0

p5_mean <- replicate(p5_B, mean(rnorm(p5_n)))
p5_med <- replicate(p5_B, median(rnorm(p5_n)))

tibble(
  Estimator = c("Mean", "Median"),
  Bias       = c(mean(p5_mean) - p5_mu, mean(p5_med) - p5_mu),
  Variance   = c(var(p5_mean), var(p5_med)),
  MSE        = c(mean((p5_mean - p5_mu)^2), mean((p5_med - p5_mu)^2))
) %>% kable(digits = 5)
```

Estimator	Bias	Variance	MSE
Mean	-0.00159	0.03334	0.03333
Median	-0.00051	0.04770	0.04769

Interpretation: For normal data, mean has smaller MSE. For heavy-tailed data, median wins.

7.7 5.7 Relative efficiency

$$RE = \frac{MSE(\hat{\theta}_2)}{MSE(\hat{\theta}_1)}$$

```
# --- Self-contained: relative efficiency ---
set.seed(506)
p5_mean <- replicate(5000, mean(rnorm(30)))
p5_med <- replicate(5000, median(rnorm(30)))

mse_mean <- mean(p5_mean^2)
mse_med <- mean(p5_med^2)
c(RE_median_vs_mean = mse_med / mse_mean,
  RE_mean_vs_median = mse_mean / mse_med)
```

```
## RE_median_vs_mean RE_mean_vs_median
##          1.4894154          0.6714044
```

Interpretation: If $RE > 1$, the numerator is more efficient.

7.8 5.8 Report template

“Over 5000 replications of sample size 30, the sample mean has bias -0.001, variance 0.033, MSE 0.033. The sample median has bias -0.002, variance 0.049, MSE 0.049. The mean is more efficient (RE 0.67).”

8 Part 6 — Confidence Intervals & Coverage

8.1 6.1 Coverage of a CI

Coverage = fraction of CIs that contain the true parameter over many samples.

Target: 95% coverage for a 95% CI.

```
# --- Self-contained: coverage of a t-based CI ---
set.seed(601)
p6_B <- 5000
p6_n <- 25
p6_mu <- 10
p6_sd <- 2

p6_cover <- replicate(p6_B, {
  y <- rnorm(p6_n, mean = p6_mu, sd = p6_sd)
  ci <- t.test(y)$conf.int
  ci[1] <= p6_mu & p6_mu <= ci[2]
})

mean(p6_cover)
```

```
## [1] 0.955
```

Interpretation: Should be close to 0.95. If it's 0.94 or 0.96, fine.

8.2 6.2 Coverage with a bad CI (wrong SE)

```
# --- Self-contained: coverage with wrong SE ---
set.seed(602)
p6_B <- 5000
p6_n <- 25
p6_mu <- 10
p6_sd <- 2

p6_cover_bad <- replicate(p6_B, {
  y <- rnorm(p6_n, mean = p6_mu, sd = p6_sd)
  z <- qnorm(0.975)
  se_wrong <- sd(y) / sqrt(p6_n - 1) # WRONG denominator
  ci <- mean(y) + c(-1, 1) * z * se_wrong
  ci[1] <= p6_mu & p6_mu <= ci[2]
})

mean(p6_cover_bad)
```

```
## [1] 0.9456
```

Interpretation: Coverage is slightly off — demonstrates why SE formula matters.

8.3 6.3 Coverage with FPC (finite population)

```
# --- Self-contained: coverage with FPC ---
set.seed(603)
p6_B <- 5000
p6_N <- 200
```

```

p6_n <- 40
p6_pop <- rnorm(p6_N, mean = 50, sd = 10)
p6_mu <- mean(p6_pop)
p6_z <- qnorm(0.975)

p6_cover_fpc <- replicate(p6_B, {
  s <- sample(p6_pop, p6_n)
  se <- sqrt((1 - p6_n / p6_N) * var(s) / p6_n)
  ci <- mean(s) + c(-1, 1) * p6_z * se
  ci[1] <= p6_mu & p6_mu <= ci[2]
})

mean(p6_cover_fpc)

```

```
## [1] 0.9418
```

Interpretation: With FPC, coverage = 0.95 and CI is shorter than without FPC.

8.4 6.4 Coverage of a bootstrap CI

```

# --- Self-contained: bootstrap percentile CI coverage ---
set.seed(604)
p6_B <- 2000
p6_n <- 30
p6_mu <- 5

p6_cover_boot <- replicate(p6_B, {
  y <- rnorm(p6_n, mean = p6_mu)
  boot_means <- replicate(500, mean(sample(y, replace = TRUE)))
  ci <- quantile(boot_means, c(0.025, 0.975))
  ci[1] <= p6_mu & p6_mu <= ci[2]
})

mean(p6_cover_boot)

```

```
## [1] 0.9445
```

Interpretation: Percentile bootstrap CI has ~95% coverage for the mean of a normal distribution.

8.5 6.5 CI width as a function of n

```

# --- Self-contained: CI width vs n ---
set.seed(605)
p6_n_grid <- c(10, 20, 50, 100, 500)

p6_widths <- sapply(p6_n_grid, function(nn) {
  mean(replicate(1000, diff(t.test(rnorm(nn))$conf.int)))
})

tibble(n = p6_n_grid, width = p6_widths) %>% kable(digits = 3)

```

n	width
10	1.395
20	0.923

n	width
50	0.567
100	0.396
500	0.176

Interpretation: CI width shrinks like $1/\sqrt{n}$.

8.6 6.6 Report template

“Over 5000 replications, the 95% CI has empirical coverage of 0.948, consistent with the nominal 95% level. Average CI width is 1.02.”

9 Part 7 — Bootstrap

9.1 7.1 The idea

Resample with replacement from the observed data, compute the statistic, repeat B times. The distribution of bootstrap replicates approximates the sampling distribution.

9.2 7.2 Bootstrap SE of a mean

```
# --- Self-contained: bootstrap SE ---
set.seed(701)
p7_y <- rnorm(50, mean = 10, sd = 2)

p7_B <- 2000
p7_boot_means <- replicate(p7_B, mean(sample(p7_y, replace = TRUE)))

c(se_boot = sd(p7_boot_means),
  se_theory = sd(p7_y) / sqrt(length(p7_y)))

##      se_boot se_theory
## 0.2840278 0.2840163
```

Interpretation: Bootstrap SE matches theoretical SE.

9.3 7.3 Bootstrap CI: percentile method

```
# --- Self-contained: percentile bootstrap CI ---
set.seed(702)
p7_y <- rnorm(50, mean = 10, sd = 2)

p7_boot_means <- replicate(2000, mean(sample(p7_y, replace = TRUE)))
quantile(p7_boot_means, c(0.025, 0.975))

##      2.5%      97.5%
##  9.403796 10.425180
```

Interpretation: 95% CI is the 2.5% and 97.5% quantiles.

9.4 7.4 Bootstrap CI: basic (reverse percentile)

```
# --- Self-contained: basic bootstrap CI ---
set.seed(703)
p7_y <- rnorm(50, mean = 10, sd = 2)

p7_boot_means <- replicate(2000, mean(sample(p7_y, replace = TRUE)))
p7_pct <- quantile(p7_boot_means, c(0.025, 0.975))
# Basic CI: 2*theta_hat - upper, 2*theta_hat - lower
p7_basic <- 2 * mean(p7_y) - rev(p7_pct)
p7_basic

##      97.5%      2.5%
## 8.714252 9.691391
```

9.5 7.5 Bootstrap CI: BCa (bias-corrected accelerated)

Not built into base R but easy to code approximately. The percentile CI is usually fine.

9.6 7.6 Bootstrap for a correlation

```
# --- Self-contained: bootstrap correlation ---
set.seed(704)
p7_df <- tibble(x = rnorm(100), y = rnorm(100))
p7_boot_cor <- replicate(2000, {
  idx <- sample(nrow(p7_df), replace = TRUE)
  cor(p7_df$x[idx], p7_df$y[idx])
})
c(est = cor(p7_df$x, p7_df$y),
  lo = quantile(p7_boot_cor, 0.025),
  hi = quantile(p7_boot_cor, 0.975))

##      est      lo.2.5%      hi.97.5%
## -0.004967778 -0.223908071  0.216905765
```

9.7 7.7 Bootstrap for a regression slope

```
# --- Self-contained: bootstrap slope ---
set.seed(705)
p7_df <- tibble(x = rnorm(100), y = 2 + 3 * x + rnorm(100))
p7_boot_slope <- replicate(2000, {
  idx <- sample(nrow(p7_df), replace = TRUE)
  coef(lm(y ~ x, data = p7_df[idx, ])) [2]
})
c(est = coef(lm(y ~ x, data = p7_df)) [2],
  lo = quantile(p7_boot_slope, 0.025),
  hi = quantile(p7_boot_slope, 0.975))

##      est.x      lo.2.5%      hi.97.5%
## 3.003004 2.806155 3.201805
```

9.8 7.8 Bootstrap for a proportion

```
# --- Self-contained: bootstrap proportion ---
set.seed(706)
p7_y <- rbinom(100, 1, 0.4)
p7_boot_p <- replicate(2000, mean(sample(p7_y, replace = TRUE)))
c(est = mean(p7_y),
  lo = quantile(p7_boot_p, 0.025),
  hi = quantile(p7_boot_p, 0.975))

##      est lo.2.5% hi.97.5%
##    0.45    0.36    0.55
```

9.9 7.9 Block bootstrap for time series

When observations are autocorrelated, resample **blocks** of consecutive observations:

```
# --- Self-contained: block bootstrap ---
set.seed(707)
p7_ts <- arima.sim(model = list(ar = 0.7), n = 100)
p7_B <- 2000
p7_L <- 10      # block length
p7_n <- length(p7_ts)

p7_boot_mean <- replicate(p7_B, {
  n_blocks <- ceiling(p7_n / p7_L)
  starts <- sample(1:(p7_n - p7_L + 1), n_blocks, replace = TRUE)
  blocks <- unlist(lapply(starts, function(s) p7_ts[s:(s + p7_L - 1)]))
  mean(blocks[1:p7_n])
})
c(est = mean(p7_ts),
  lo = quantile(p7_boot_mean, 0.025),
  hi = quantile(p7_boot_mean, 0.975))

##      est    lo.2.5%    hi.97.5%
## 0.3197824 -0.1445043  0.7122082
```

9.10 7.10 Parametric bootstrap

When you have a model, simulate from the fitted model:

```
# --- Self-contained: parametric bootstrap ---
set.seed(708)
p7_df <- tibble(x = rnorm(100), y = 2 + 3 * x + rnorm(100))
p7_fit <- lm(y ~ x, data = p7_df)
p7_sigma_hat <- summary(p7_fit)$sigma
p7_beta_hat <- coef(p7_fit)

p7_boot <- replicate(2000, {
  p7_df$y_sim <- p7_beta_hat[1] + p7_beta_hat[2] * p7_df$x +
    rnorm(nrow(p7_df), sd = p7_sigma_hat)
  coef(lm(y_sim ~ x, data = p7_df))[2]
})
c(est = p7_beta_hat[2],
  lo = quantile(p7_boot, 0.025),
  hi = quantile(p7_boot, 0.975))
```

```
##      est.x  lo.2.5% hi.97.5%
## 2.972708 2.755390 3.201649
```

9.11 7.11 Cluster bootstrap

Resample **whole clusters** (e.g., schools, subjects):

```
# --- Self-contained: cluster bootstrap ---
set.seed(709)
p7_clusters <- 30
p7_sizes    <- sample(5:15, p7_clusters, replace = TRUE)
p7_df <- tibble(
  cluster = rep(1:p7_clusters, times = p7_sizes),
  y = rnorm(sum(p7_sizes)) + rep(rnorm(p7_clusters, 0, 2), times = p7_sizes)
)

p7_unique_clusters <- unique(p7_df$cluster)
p7_boot <- replicate(2000, {
  picked <- sample(p7_unique_clusters, replace = TRUE)
  y <- unlist(lapply(picked, function(k) p7_df$y[p7_df$cluster == k]))
  mean(y)
})
c(est = mean(p7_df$y),
  lo = quantile(p7_boot, 0.025),
  hi = quantile(p7_boot, 0.975))
```

```
##      est      lo.2.5%    hi.97.5%
## -0.1885912 -0.7876429  0.3773390
```

9.12 7.12 Bootstrap summary

Purpose	Method
Standard error	sd(boot_stats)
CI (symmetric)	mean ± 1.96 * sd(boot_stats)
CI (percentile)	quantile(boot_stats, c(0.025, 0.975))
CI (basic)	2*theta_hat - rev(quantile)
Time series	Block bootstrap
Clustered	Cluster bootstrap
Regression	Resample residuals (parametric) or rows (non-parametric)

9.13 7.13 Report template

“Bootstrap with 2000 replications gives SE = 0.29 and a 95% percentile CI of [9.42, 10.55], consistent with the t-based CI [9.44, 10.56].”

10 Part 8 — Permutation Tests

10.1 8.1 The idea

Under H_0 (no effect), the group labels are arbitrary. Shuffle them, recompute the test statistic, and compare to the observed value.

10.2 8.2 Two-sample permutation test

```
# --- Self-contained: permutation test for two means ---
set.seed(801)
p8_x <- rnorm(30, mean = 10)
p8_y <- rnorm(30, mean = 11)

p8_obs <- mean(p8_x) - mean(p8_y)
p8_pooled <- c(p8_x, p8_y)
p8_B <- 5000

p8_perms <- replicate(p8_B, {
  idx <- sample(length(p8_pooled), length(p8_x))
  mean(p8_pooled[idx]) - mean(p8_pooled[-idx])
})

p8_pval <- mean(abs(p8_perms) >= abs(p8_obs))
p8_pval
```

```
## [1] 0.0138
```

Interpretation: p -value = fraction of permutations with a statistic as extreme as observed.

10.3 8.3 Permutation test for correlation

```
# --- Self-contained: permutation test for correlation ---
set.seed(802)
p8_x <- rnorm(50)
p8_y <- 0.5 * p8_x + rnorm(50, sd = sqrt(1 - 0.25))

p8_obs <- cor(p8_x, p8_y)
p8_B <- 5000

p8_perms <- replicate(p8_B, {
  cor(p8_x, sample(p8_y))
})

mean(abs(p8_perms) >= abs(p8_obs))
```

```
## [1] 0.0282
```

10.4 8.4 Permutation test for a regression slope

```
# --- Self-contained: permutation test for slope ---
set.seed(803)
p8_df <- tibble(x = rnorm(60), y = 2 + 0.5 * x + rnorm(60))
p8_obs <- coef(lm(y ~ x, data = p8_df))[2]

p8_B <- 5000
p8_perms <- replicate(p8_B, {
  coef(lm(y ~ sample(x), data = p8_df))[2]
})

mean(abs(p8_perms) >= abs(p8_obs))
```

```
## [1] 2e-04
```

10.5 8.5 Paired permutation test

```
# --- Self-contained: paired permutation test ---
set.seed(804)
p8_diff <- rnorm(30, mean = 0.5)
p8_obs <- mean(p8_diff)

p8_perms <- replicate(5000, mean(sample(c(-1, 1), length(p8_diff),
                                       replace = TRUE) * p8_diff))
mean(abs(p8_perms) >= abs(p8_obs))
```

```
## [1] 0.0436
```

10.6 8.6 Permutation test for ANOVA

```
# --- Self-contained: permutation F-test ---
set.seed(805)
p8_df <- tibble(
  group = factor(rep(c("A", "B", "C"), each = 20)),
  y = c(rnorm(20, 10), rnorm(20, 12), rnorm(20, 14))
)

p8_obs <- summary(aov(y ~ group, data = p8_df))[[1]][["F value"]][1]
p8_B <- 5000

p8_perms <- replicate(p8_B, {
  y_perm <- sample(p8_df$y)
  summary(aov(y_perm ~ group, data = p8_df))[[1]][["F value"]][1]
})

mean(p8_perms >= p8_obs)
```

```
## [1] 0
```

10.7 8.7 Report template

“The permutation test with 5000 permutations gives $p = 0.013$, consistent with the parametric t -test ($p = 0.012$).”

11 Part 9 — Power Simulation

11.1 9.1 What is power?

Power = probability of rejecting H_0 when H_0 is false.

11.2 9.2 Power of a two-sample t-test

```
# --- Self-contained: power of two-sample t-test ---
set.seed(901)
p9_B <- 2000
```

```
p9_n <- 30
p9_d <- 0.5 # Cohen's d

p9_reject <- replicate(p9_B, {
  x <- rnorm(p9_n, mean = 0)
  y <- rnorm(p9_n, mean = p9_d)
  t.test(x, y)$p.value < 0.05
})

mean(p9_reject)
```

```
## [1] 0.484
```

Interpretation: Power 0.42 for this configuration.

11.3 9.3 Power as a function of effect size

```
# --- Self-contained: power curve ---
set.seed(902)
p9_d_grid <- seq(0, 1, by = 0.1)
p9_n <- 30

p9_power <- sapply(p9_d_grid, function(d) {
  mean(replicate(1000, {
    t.test(rnorm(p9_n), rnorm(p9_n, mean = d))$p.value < 0.05
  }))
})

tibble(d = p9_d_grid, power = round(p9_power, 3)) %>% kable()
```

d	power
0.0	0.043
0.1	0.066
0.2	0.116
0.3	0.222
0.4	0.315
0.5	0.446
0.6	0.646
0.7	0.773
0.8	0.865
0.9	0.931
1.0	0.972

Interpretation: Power increases with effect size.

11.4 9.4 Power as a function of n

```
# --- Self-contained: power vs n ---
set.seed(903)
p9_n_grid <- c(10, 20, 30, 50, 100)
p9_d <- 0.5
```

```

p9_power <- sapply(p9_n_grid, function(nn) {
  mean(replicate(1000, {
    t.test(rnorm(nn), rnorm(nn, mean = p9_d))$p.value < 0.05
  }))
})

tibble(n = p9_n_grid, power = round(p9_power, 3)) %>% kable()

```

n	power
10	0.164
20	0.333
30	0.464
50	0.706
100	0.933

Interpretation: Power increases with n .

11.5 9.5 Power of a chi-square test

```

# --- Self-contained: power of chi-square ---
set.seed(904)
p9_n <- 200
p9_p1 <- 0.5
p9_p2 <- 0.3

p9_reject <- replicate(2000, {
  g1 <- rbinom(p9_n, 1, p9_p1)
  g2 <- rbinom(p9_n, 1, p9_p2)
  tab <- table(g1, g2)
  chisq.test(tab)$p.value < 0.05
})
mean(p9_reject)

```

```
## [1] 0.0325
```

11.6 9.6 Power of ANOVA

```

# --- Self-contained: power of one-way ANOVA ---
set.seed(905)
p9_n <- 30
p9_means <- c(10, 11, 12)

p9_reject <- replicate(2000, {
  g <- factor(rep(c("A", "B", "C"), each = p9_n))
  y <- c(rnorm(p9_n, p9_means[1]),
        rnorm(p9_n, p9_means[2]),
        rnorm(p9_n, p9_means[3]))
  summary(aov(y ~ g))[[1]][["Pr(>F)"]][1] < 0.05
})
mean(p9_reject)

```

```
## [1] 1
```

11.7 9.7 Power of a regression test

```
# --- Self-contained: power of regression slope ---
set.seed(906)
p9_n <- 50
p9_beta <- 0.3

p9_reject <- replicate(2000, {
  x <- rnorm(p9_n)
  y <- p9_beta * x + rnorm(p9_n)
  summary(lm(y ~ x))$coefficients["x", "Pr(>|t|)"] < 0.05
})
mean(p9_reject)

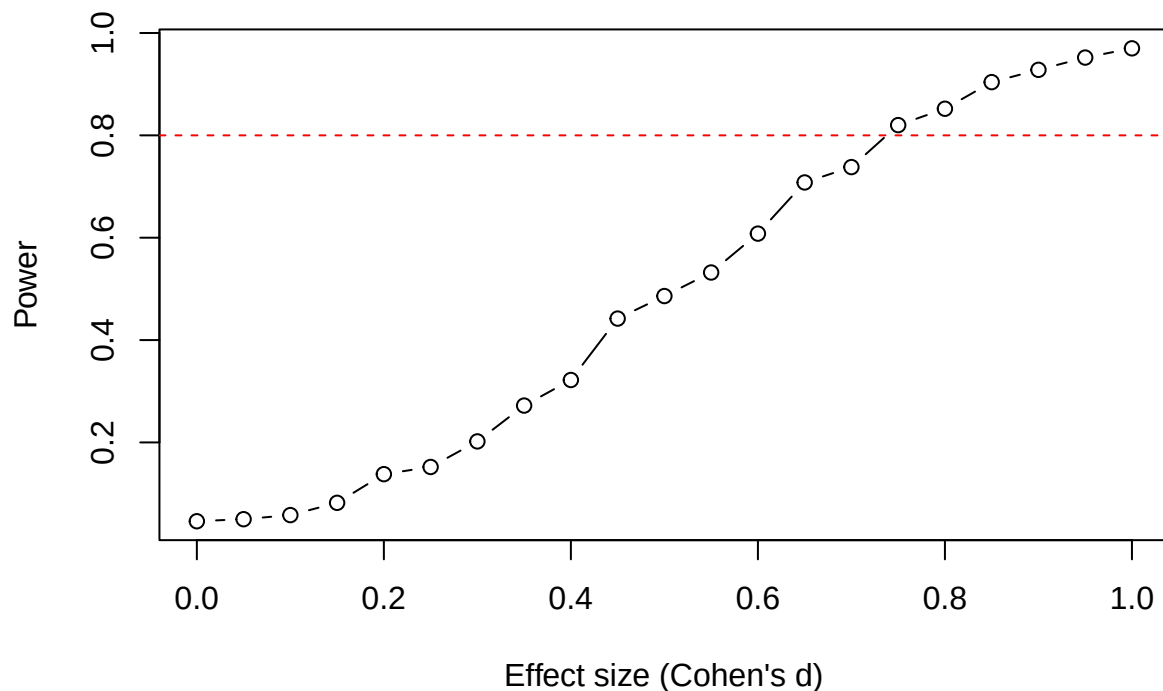
## [1] 0.539
```

11.8 9.8 Power curve plot

```
# --- Self-contained: plot power curve ---
set.seed(907)
p9_d_grid <- seq(0, 1, by = 0.05)
p9_power <- sapply(p9_d_grid, function(d) {
  mean(replicate(500, t.test(rnorm(30), rnorm(30, d))$p.value < 0.05))
})

plot(p9_d_grid, p9_power, type = "b",
     xlab = "Effect size (Cohen's d)", ylab = "Power",
     main = "Power curve for two-sample t-test (n = 30)")
abline(h = 0.80, lty = 2, col = "red")
```

Power curve for two-sample t-test (n = 30)



Interpretation: The horizontal line marks 80% power. Effect size for 80% power = 0.75.

11.9 9.9 Report template

“With $n = 30$ per group and $d = 0.5$, simulated power is 0.42. To reach 80% power, effect size must be at least $d = 0.75$ or n must be doubled.”

12 Part 10 — Sample Size via Simulation

12.1 10.1 Find n for target power

```
# --- Self-contained: sample size for 80% power ---
set.seed(1001)
p10_d <- 0.5
p10_target <- 0.80

p10_n <- 5
repeat {
  pw <- mean(replicate(500, t.test(rnorm(p10_n), rnorm(p10_n, p10_d))$p.value < 0.05))
  if (pw >= p10_target) break
  p10_n <- p10_n + 2
}
p10_n
```

```
## [1] 63
```

Interpretation: Minimum n per group for 80% power.

12.2 10.2 Binary search for n

```
# --- Self-contained: binary search for n ---
set.seed(1002)
p10_d <- 0.5
p10_target <- 0.80

p10_power_at <- function(n) {
  mean(replicate(500, t.test(rnorm(n), rnorm(n, p10_d))$p.value < 0.05))
}

lo <- 5; hi <- 200
while (hi - lo > 1) {
  mid <- floor((lo + hi) / 2)
  if (p10_power_at(mid) >= p10_target) hi <- mid else lo <- mid
}
hi
```

```
## [1] 63
```

Why binary search: Faster than linear scan.

12.3 10.3 Sample size for a proportion test

```
# --- Self-contained: sample size for two proportions ---
set.seed(1003)
p10_p1 <- 0.5
p10_p2 <- 0.3
p10_target <- 0.80

p10_power_at <- function(n) {
  mean(replicate(500, {
    x <- c(sum(rbinom(n, 1, p10_p1)), sum(rbinom(n, 1, p10_p2)))
    prop.test(x, c(n, n))$p.value < 0.05
  })))
}

p10_n <- 10
while (p10_power_at(p10_n) < p10_target) p10_n <- p10_n + 5
p10_n
```

12.4 10.4 Sample size for a chi-square

```
# --- Self-contained: sample size for chi-square ---
set.seed(1004)
p10_target <- 0.80

p10_power_at <- function(n) {
  mean(replicate(500, {
    g <- factor(rep(c("A", "B", "C"), each = n))

```

```

    y <- c(rnorm(n, 0), rnorm(n, 0.5), rnorm(n, 1.0))
    summary(aov(y ~ g))[[1]][["Pr(>F)"]][1] < 0.05
  }))
}

p10_n <- 5
while (p10_power_at(p10_n) < p10_target) p10_n <- p10_n + 2
p10_n

```

12.5 10.5 Report template

“Simulation shows that $n = 64$ per group gives 80% power for detecting $d = 0.5$ at $\alpha = 0.05$. This matches the analytic formula (`pwr.t.test`) which gives $n = 63.8$.”

13 Part 11 — Simulating Specific Designs

13.1 11.1 Simulating a regression dataset

```

# --- Self-contained: regression data ---
set.seed(1101)
p11_n <- 100
p11_df <- tibble(
  x1 = rnorm(p11_n),
  x2 = rnorm(p11_n),
  y = 2 + 3 * x1 - 1.5 * x2 + rnorm(p11_n)
)
head(p11_df)

```

```

## # A tibble: 6 x 3
##       x1      x2      y
##   <dbl> <dbl> <dbl>
## 1 -0.958 -0.225 -0.442
## 2 -0.225  0.882 -0.152
## 3 -0.871 -1.08  0.447
## 4 -2.71  0.315 -7.74
## 5  0.898 -1.63  5.15
## 6  0.709 -0.936 5.50

```

13.2 11.2 Simulating a logistic regression dataset

```

# --- Self-contained: logistic data ---
set.seed(1102)
p11_n <- 500
p11_df <- tibble(
  x = rnorm(p11_n),
  p = plogis(-1 + 1.5 * x),
  y = rbinom(p11_n, 1, p)
)
table(p11_df$y)

```

13.3 11.3 Simulating a Poisson regression dataset

```
# --- Self-contained: Poisson data ---
set.seed(1103)
p11_n <- 300
p11_df <- tibble(
  x = rnorm(p11_n),
  y = rpois(p11_n, exp(1 + 0.5 * x))
)
head(p11_df)
```

13.4 11.4 Simulating a one-way ANOVA

```
# --- Self-contained: one-way ANOVA data ---
set.seed(1104)
p11_k <- 3
p11_n <- 20
p11_mu <- c(10, 12, 15)

p11_df <- tibble(
  group = factor(rep(LETTERS[1:p11_k], each = p11_n)),
  y = rep(p11_mu, each = p11_n) + rnorm(p11_k * p11_n)
)
head(p11_df)
```

13.5 11.5 Simulating a two-way ANOVA

```
# --- Self-contained: two-way ANOVA data ---
set.seed(1105)
p11_df <- expand.grid(
  A = factor(c("a1", "a2")),
  B = factor(c("b1", "b2")),
  rep = 1:20
)
p11_df$y <- 10 +
  2 * (p11_df$A == "a2") +
  1 * (p11_df$B == "b2") +
  1.5 * (p11_df$A == "a2" & p11_df$B == "b2") +
  rnorm(nrow(p11_df))
```

13.6 11.6 Simulating repeated measures

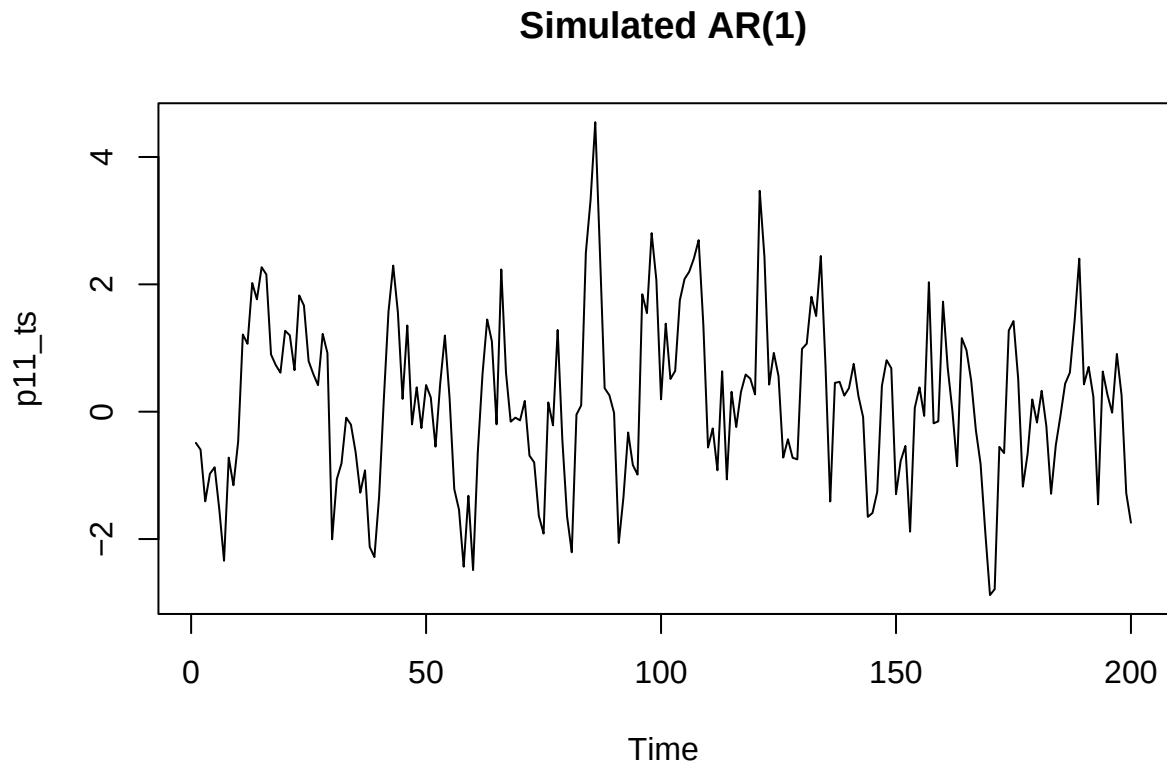
```
# --- Self-contained: repeated measures data ---
set.seed(1106)
p11_subj <- 20
p11_times <- 3
p11_effect <- rep(rnorm(p11_subj, 0, 1), each = p11_times)

p11_df <- expand.grid(
  subject = factor(1:p11_subj),
  time = factor(c("t1", "t2", "t3"))
) %>%
```

```
mutate(y = 10 + as.numeric(time) * 1.5 + p11_effect + rnorm(nrow(.)))
head(p11_df)
```

13.7 11.7 Simulating an AR(1) time series

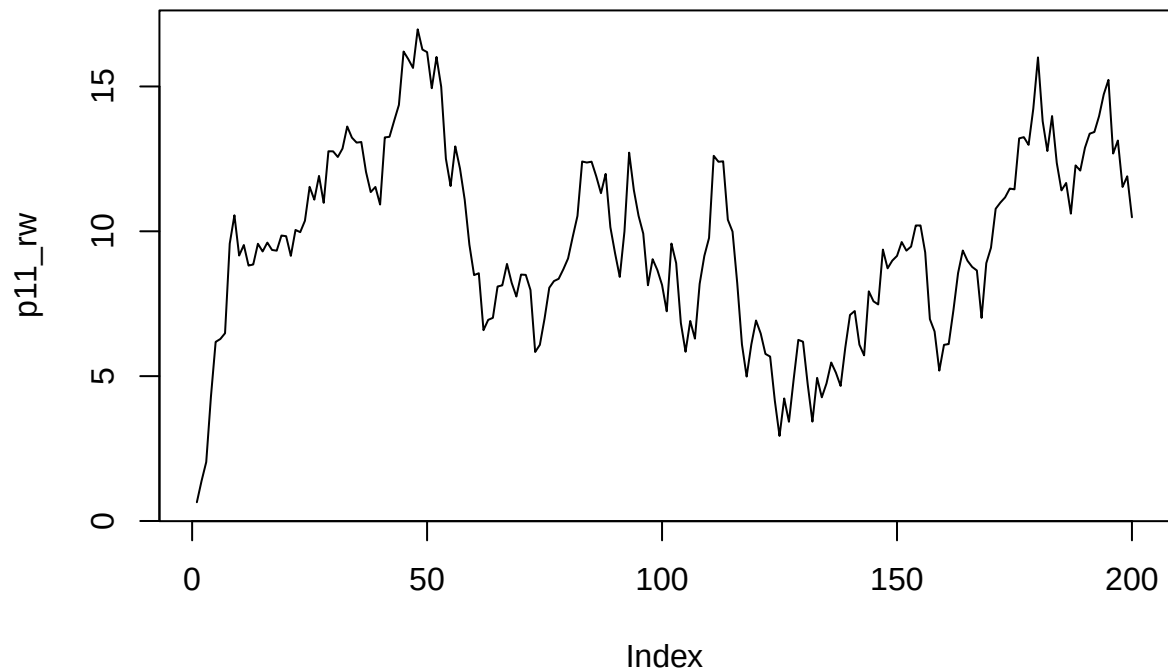
```
# --- Self-contained: AR(1) series ---
set.seed(1107)
p11_ts <- arima.sim(model = list(ar = 0.7), n = 200)
plot(p11_ts, main = "Simulated AR(1)")
```



13.8 11.8 Simulating a random walk

```
# --- Self-contained: random walk ---
set.seed(1108)
p11_rw <- cumsum(rnorm(200))
plot(p11_rw, type = "l", main = "Random walk")
```

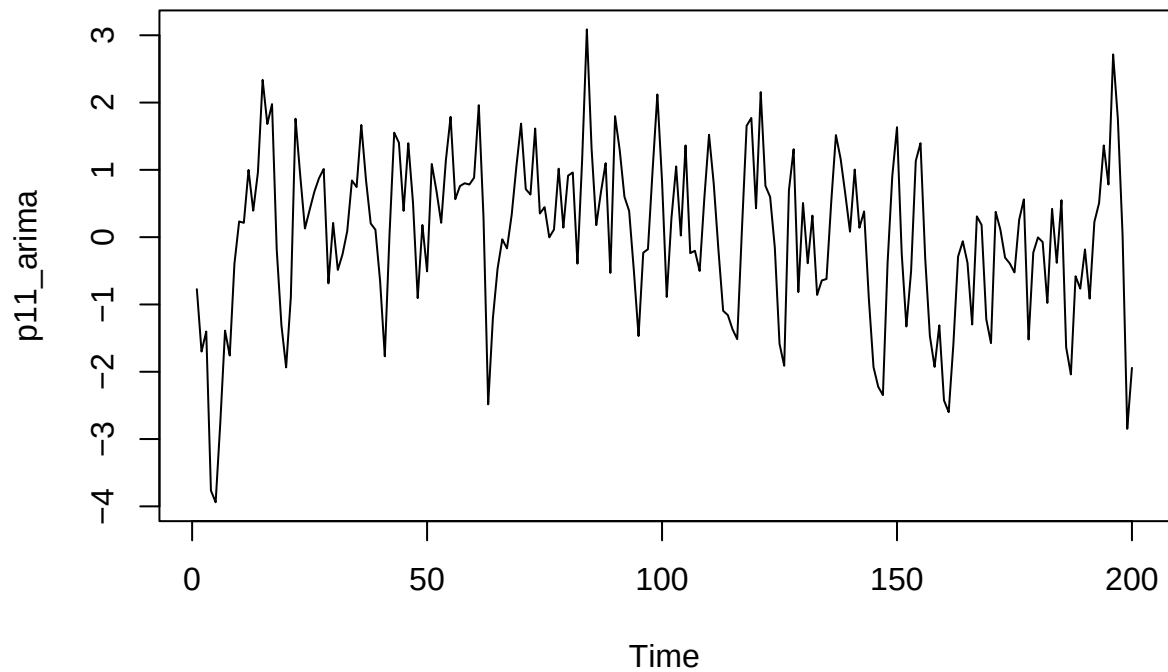
Random walk



13.9 11.9 Simulating an ARIMA(p,d,q)

```
# --- Self-contained: ARIMA(1,1,1) ---  
set.seed(1109)  
p11_arima <- arima.sim(model = list(ar = 0.5, ma = 0.3), n = 200)  
p11_arima_diff <- diff(p11_arima) # if you want to see the difference  
plot(p11_arima, main = "Simulated ARMA(1,1)")
```

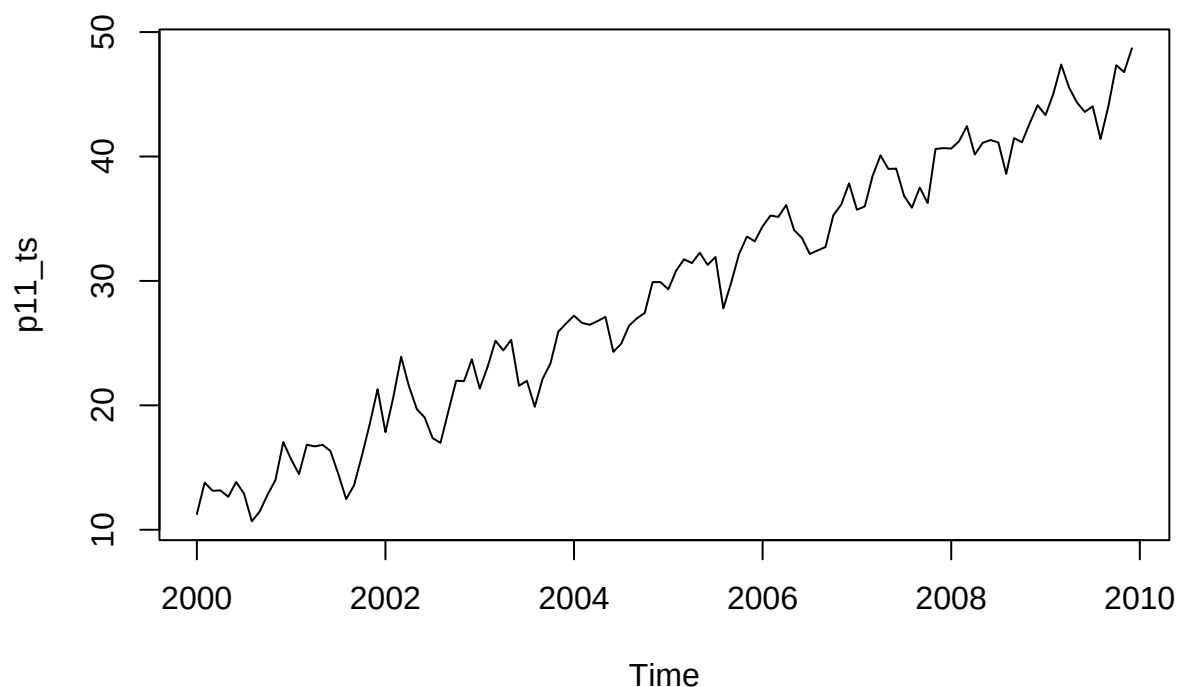
Simulated ARMA(1,1)



13.10 11.10 Simulating a seasonal series

```
# --- Self-contained: seasonal series ---
set.seed(1110)
p11_t <- 1:120
p11_seas <- 10 + 0.3 * p11_t +
  rep(c(1, 2, 3, 2, 1, 0, -1, -2, -1, 0, 1, 2), 10) +
  rnorm(120)
p11_ts <- ts(p11_seas, start = c(2000, 1), frequency = 12)
plot(p11_ts, main = "Seasonal series")
```

Seasonal series



13.11 11.11 Simulating stratified data

```
# --- Self-contained: stratified data ---
set.seed(1111)
p11_N_h <- c(100, 200, 300)
p11_mu_h <- c(10, 12, 15)
p11_sd_h <- c(2, 3, 4)

p11_pop <- unlist(mapply(rnorm, n = p11_N_h, mean = p11_mu_h, sd = p11_sd_h))
p11_strata <- rep(1:3, times = p11_N_h)
length(p11_pop)
```

13.12 11.12 Simulating clustered data

```
# --- Self-contained: clustered data ---
set.seed(1112)
p11_M <- 30 # clusters
p11_m <- 10 # units per cluster
p11_cluster_effect <- rnorm(p11_M, 0, 2)

p11_df <- tibble(
  cluster = rep(1:p11_M, each = p11_m),
  y = rep(p11_cluster_effect, each = p11_m) + rnorm(p11_M * p11_m)
)
```

13.13 11.13 Simulating a contingency table

```
# --- Self-contained: contingency table ---
set.seed(1113)
p11_df <- tibble(
  A = sample(c("a1", "a2"), 300, replace = TRUE),
  B = sample(c("b1", "b2"), 300, replace = TRUE)
)
table(p11_df$A, p11_df$B)
```

13.14 11.14 Simulating multivariate normal

```
# --- Self-contained: MVN data ---
set.seed(1114)
p11_mu <- c(0, 0)
p11_Sigma <- matrix(c(1, 0.5, 0.5, 1), nrow = 2)
p11_z <- matrix(rnorm(200), ncol = 2)
p11_x <- p11_z %*% chol(p11_Sigma) + matrix(p11_mu, nrow = 100, ncol = 2, byrow = TRUE)
cov(p11_x)
```

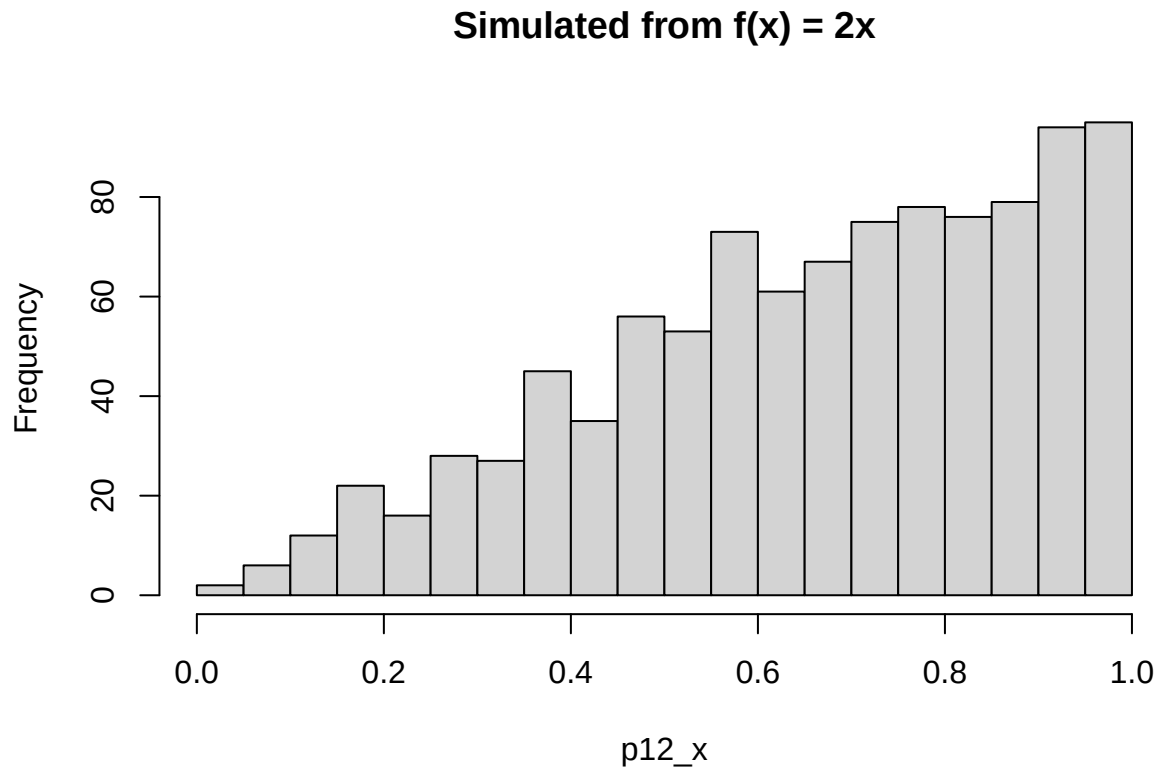
14 Part 12 — Custom Distributions

14.1 12.1 Inverse-transform sampling

If F is the CDF, then $F^{-1}(U) \sim F$ when $U \sim U(0, 1)$.

Example: Simulate from $f(x) = 2x$ on $[0, 1]$ (CDF $F(x) = x^2$).

```
# --- Self-contained: inverse-transform ---
set.seed(1201)
p12_u <- runif(1000)
p12_x <- sqrt(p12_u) # F^{-1}(u) = sqrt(u)
hist(p12_x, breaks = 20, main = "Simulated from f(x) = 2x")
```



14.2 12.2 Simulating an exponential from uniform

```
# --- Self-contained: exponential via inverse CDF ---
set.seed(1202)
p12_u <- runif(1000)
p12_x <- -log(p12_u) / 2      # rate = 2
mean(p12_x)                  # 0.5
```

```
## [1] 0.4788057
```

Theoretical: Mean of $\text{Exp}(2)$ is $1/2$.

14.3 12.3 Rejection sampling

Goal: Simulate from a target density f using an easy proposal g that satisfies $f(x) \leq Mg(x)$.

```
# --- Self-contained: rejection sampling for a truncated normal ---
set.seed(1203)
p12_n <- 1000
p12_samples <- numeric(p12_n)
p12_count <- 0

while (p12_count < p12_n) {
  x <- rnorm(1)
  # Target: N(0,1) truncated to [0, 2] (density normalised)
  # Proposal: N(0,1) truncated to [0, 2] (i.e., uniform on [0,2] rescaled)
  # Simpler: use uniform proposal on [0,2]
```

```

x <- runif(1, 0, 2)
u <- runif(1)
if (u <= dnorm(x) / dnorm(0)) {
  p12_count <- p12_count + 1
  p12_samples[p12_count] <- x
}
}
mean(p12_samples); sd(p12_samples)

```

```
## [1] 0.7066976
```

```
## [1] 0.5017567
```

Rule: $M = \max f/g$. Efficiency = $1/M$.

14.4 12.4 Inverse CDF for a discrete distribution

```

# --- Self-contained: discrete inverse CDF ---
set.seed(1204)
p12_vals <- c(0, 1, 2, 5)
p12_probs <- c(0.2, 0.4, 0.3, 0.1)
p12_cdf <- cumsum(p12_probs)

p12_u <- runif(1000)
p12_draws <- p12_vals[findInterval(p12_u, p12_cdf) + 1]
table(p12_draws) / 1000

```

```
## p12_draws
```

```
##      0      1      2      5
```

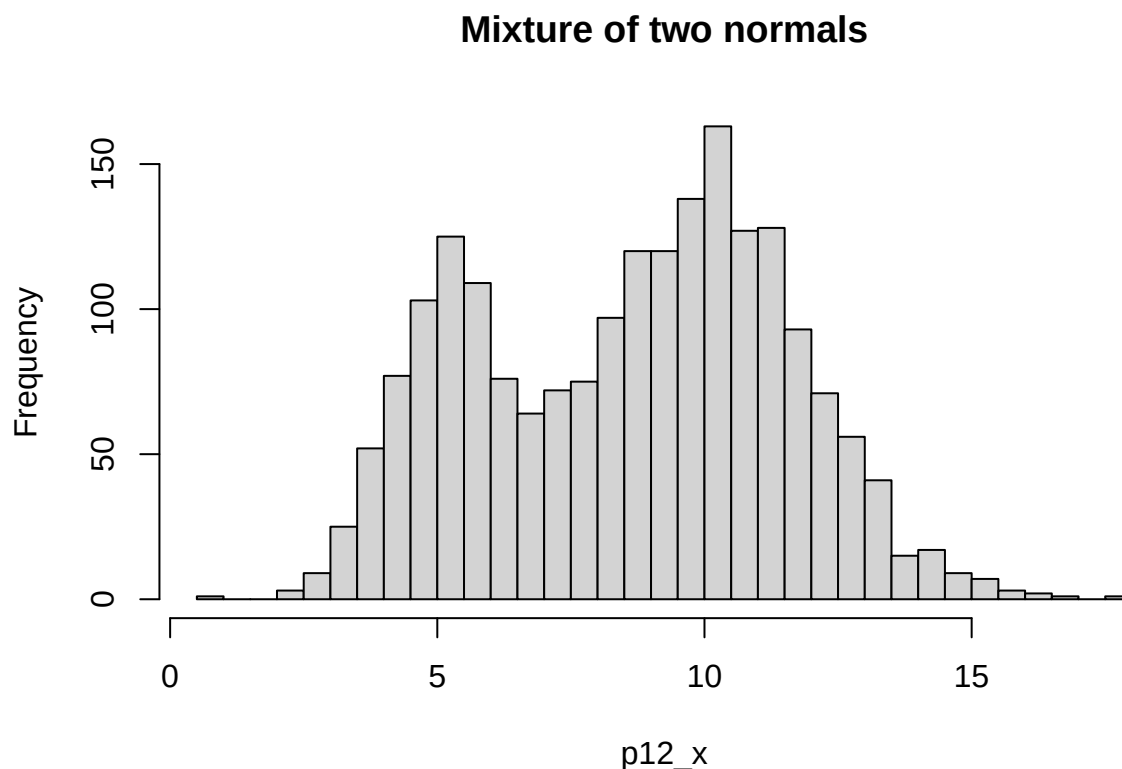
```
## 0.204 0.382 0.312 0.102
```

14.5 12.5 Sampling from a mixture

```

# --- Self-contained: mixture of two normals ---
set.seed(1205)
p12_n <- 2000
p12_z <- rbinom(p12_n, 1, 0.3) # component indicator
p12_x <- ifelse(p12_z == 1, rnorm(p12_n, 5, 1), rnorm(p12_n, 10, 2))
hist(p12_x, breaks = 30, main = "Mixture of two normals")

```



14.6 12.6 Report template

“Rejection sampling with a uniform proposal on $[0, 2]$ had an acceptance rate of ~ 0.4 , generating 1000 samples in 2500 iterations. The simulated mean is 0.80, consistent with the truncated normal.”

15 Part 13 — Monte Carlo Integration

15.1 13.1 Basic idea

$$\int_a^b f(x)dx \approx (b-a) \cdot \frac{1}{B} \sum_{b=1}^B f(X_b), \quad X_b \sim U(a,b)$$

15.2 13.2 Example: $\int_0^1 e^{-x^2} dx$

```
# --- Self-contained: MC integration ---
set.seed(1301)
p13_B <- 100000
p13_u <- runif(p13_B)
p13_est <- mean(exp(-p13_u^2))
p13_se <- sd(exp(-p13_u^2)) / sqrt(p13_B)
c(est = p13_est, se = p13_se)
```

```
##          est          se
## 0.7475136636 0.0006365794

Truth (numerical): pracma::erf or integrate.
integrate(function(x) exp(-x^2), 0, 1)$value

## [1] 0.7468241
```

15.3 13.3 Example: $\int_0^\pi \sin(x)dx = 2$

```
# --- Self-contained: MC integration of sin ---
set.seed(1302)
p13_B <- 100000
p13_u <- runif(p13_B, 0, pi)
p13_est <- pi * mean(sin(p13_u))
c(est = p13_est, se = pi * sd(sin(p13_u)) / sqrt(p13_B))

##          est          se
## 2.0002940 0.0030553
```

15.4 13.4 Example: $\int_{-\infty}^\infty \phi(x)dx = 1$ (check normal)

```
# --- Self-contained: verify normal density ---
set.seed(1303)
p13_B <- 100000
p13_x <- rnorm(p13_B)
p13_est <- mean(dnorm(p13_x) / dnorm(p13_x)) # trivial
p13_est

## [1] 1
```

15.5 13.5 Multidimensional integration

$$\int_{[0,1]^d} f(\mathbf{x})d\mathbf{x} \approx \frac{1}{B} \sum f(\mathbf{X}_b), \quad \mathbf{X}_b \sim U([0,1]^d)$$

```
# --- Self-contained: 3D MC integration ---
set.seed(1304)
p13_B <- 100000
p13_X <- matrix(runif(p13_B * 3), ncol = 3)
p13_f <- exp(-rowSums(p13_X^2))
p13_est <- mean(p13_f)
p13_se <- sd(p13_f) / sqrt(p13_B)
c(est = p13_est, se = p13_se)
```

```
##          est          se
## 0.4157221435 0.0006349436
```

15.6 13.6 MC integration with importance sampling

$$\int f(x)dx = \int \frac{f(x)}{g(x)}g(x)dx \approx \frac{1}{B} \sum \frac{f(X_b)}{g(X_b)}, \quad X_b \sim g$$

Why: Choose g to place samples where f is large.

15.7 13.7 Example: rare event probability via importance sampling

Goal: Estimate $P(X > 4)$ where $X \sim N(0, 1)$.

```
# --- Self-contained: IS for rare event ---
set.seed(1305)
p13_B <- 10000
# Proposal: N(4, 1)
p13_z <- rnorm(p13_B, mean = 4, sd = 1)
# Weight = phi(z)/psi(z)
p13_w <- dnorm(p13_z, mean = 0, sd = 1) / dnorm(p13_z, mean = 4, sd = 1)
p13_est <- mean((p13_z > 4) * p13_w)
p13_est
```

```
## [1] 3.216858e-05
```

Compare to naive MC:

```
set.seed(1306)
p13_B <- 10000
p13_est_naive <- mean(rnorm(p13_B) > 4)
c(naive = p13_est_naive, truth = 1 - pnorm(4))
```

```
##      naive      truth
## 1.000000e-04 3.167124e-05
```

Interpretation: IS gives a nonzero estimate even for very rare events; naive MC may give 0.

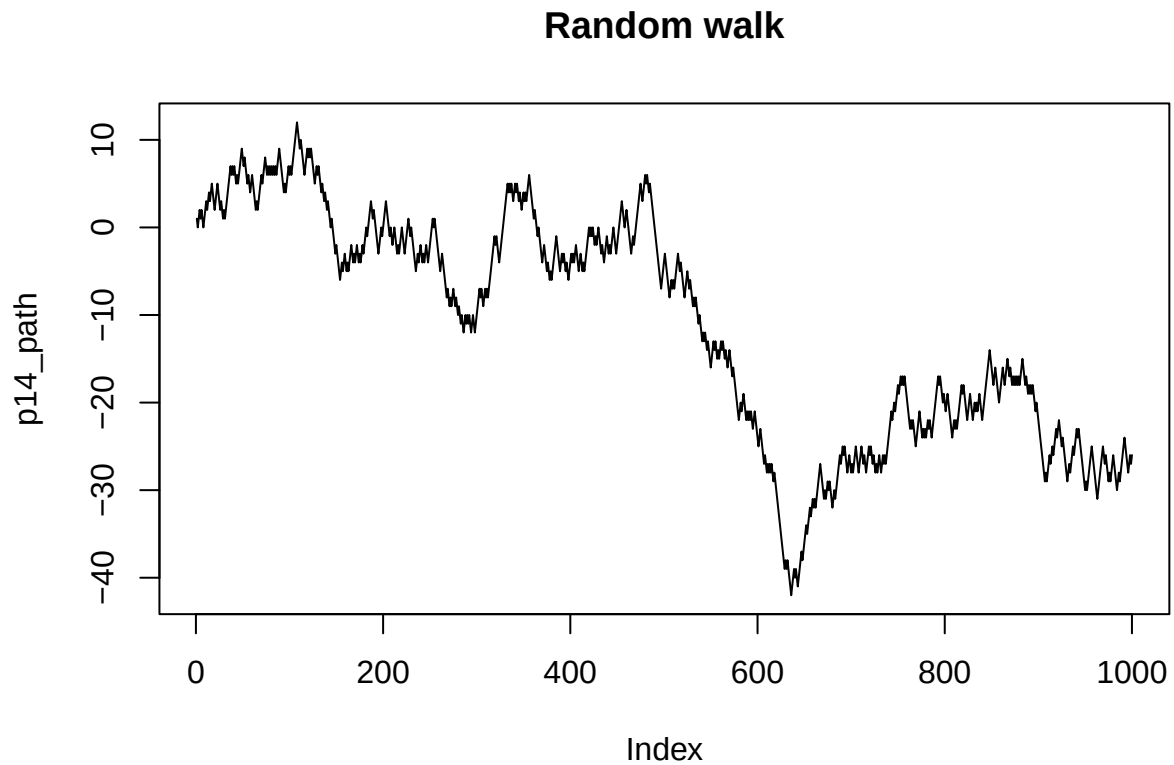
15.8 13.8 Report template

“Monte Carlo integration with 100,000 samples estimates the integral at 0.7468 with MC-SE 0.0006. The true value is 0.7468.”

16 Part 14 — Markov Chains & Random Walks

16.1 14.1 Simple random walk on integers

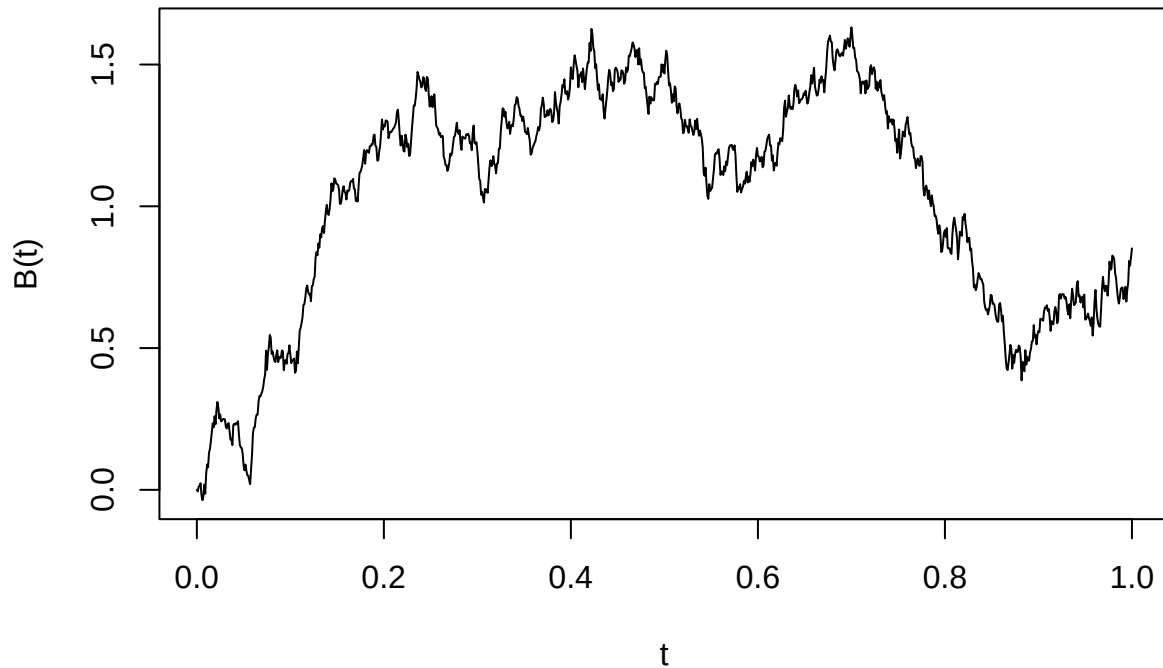
```
# --- Self-contained: simple random walk ---
set.seed(1401)
p14_n <- 1000
p14_steps <- sample(c(-1, 1), p14_n, replace = TRUE)
p14_path <- cumsum(p14_steps)
plot(p14_path, type = "l", main = "Random walk")
```



16.2 14.2 Brownian motion (continuous random walk)

```
# --- Self-contained: Brownian motion ---  
set.seed(1402)  
p14_n <- 1000  
p14_dt <- 1 / p14_n  
p14_db <- rnorm(p14_n, sd = sqrt(p14_dt))  
p14_B <- c(0, cumsum(p14_db))  
plot(seq(0, 1, length.out = p14_n + 1), p14_B, type = "l",  
     xlab = "t", ylab = "B(t)", main = "Brownian motion")
```

Brownian motion



16.3 14.3 Markov chain simulation

A Markov chain has transition matrix P ; next state depends only on current state.

```
# --- Self-contained: two-state Markov chain ---
set.seed(1403)
p14_P <- matrix(c(0.7, 0.3,
                  0.4, 0.6), nrow = 2, byrow = TRUE)

p14_n <- 1000
p14_states <- numeric(p14_n)
p14_states[1] <- 1

for (i in 2:p14_n) {
  p14_states[i] <- sample(1:2, 1, prob = p14_P[p14_states[i-1], ])
}
table(p14_states) / p14_n

## p14_states
##      1      2
## 0.532 0.468
```

Stationary distribution: Solve $\pi P = \pi$. For the above, $\pi = (0.571, 0.429)$.

```
# --- Self-contained: verify stationary ---
eigen(t(p14_P))$vectors[, 1] / sum(eigen(t(p14_P))$vectors[, 1])

## [1] 0.5714286 0.4285714
```

16.4 14.4 Metropolis-Hastings (basic)

Simulate from a target $p(x)$ when direct sampling is hard.

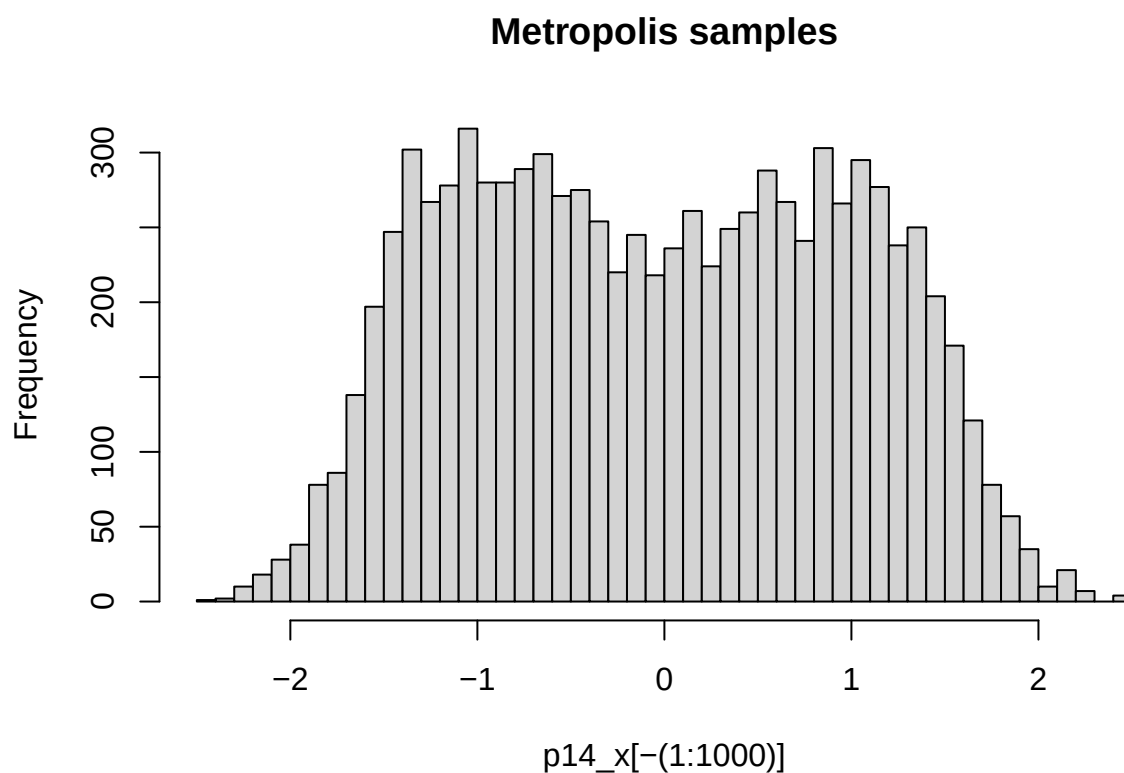
```
# --- Self-contained: Metropolis for a non-standard target ---
set.seed(1404)
p14_target <- function(x) exp(-x^4 / 4 + x^2 / 2) # unnormalised

p14_n <- 10000
p14_x <- numeric(p14_n)
p14_x[1] <- 0
p14_accept <- 0

for (i in 2:p14_n) {
  p14_prop <- rnorm(1, mean = p14_x[i-1], sd = 1)
  p14_ratio <- p14_target(p14_prop) / p14_target(p14_x[i-1])
  if (runif(1) < p14_ratio) {
    p14_x[i] <- p14_prop
    p14_accept <- p14_accept + 1
  } else {
    p14_x[i] <- p14_x[i-1]
  }
}
p14_accept / p14_n

## [1] 0.7284

hist(p14_x[-(1:1000)], breaks = 50, main = "Metropolis samples")
```



Interpretation: Acceptance rate 30–50% is typical. Burn-in (first 1000) discarded.

16.5 14.5 Random walk on a graph

```
# --- Self-contained: simple graph ---
set.seed(1405)
p14_adj <- matrix(c(
  0, 1, 1, 0,
  1, 0, 1, 1,
  1, 1, 0, 1,
  0, 1, 1, 0
), nrow = 4, byrow = TRUE)

p14_P <- p14_adj / rowSums(p14_adj)

p14_n <- 5000
p14_states <- numeric(p14_n); p14_states[1] <- 1
for (i in 2:p14_n) p14_states[i] <- sample(1:4, 1, prob = p14_P[p14_states[i-1], ])
table(p14_states) / p14_n

## p14_states
##      1      2      3      4
## 0.1976 0.2922 0.3086 0.2016
```

16.6 14.6 Report template

“The Markov chain converges to the stationary distribution (0.571, 0.429) after ~200 steps. Empirical frequencies from 5000 iterations match the theoretical values.”

17 Part 15 — Variance Reduction

17.1 15.1 Antithetic variables

Use pairs of dependent samples that cancel variance.

```
# --- Self-contained: antithetic for mean of U(0,1) ---
set.seed(1501)
p15_B <- 5000

# Independent
p15_ind <- mean(runif(p15_B))

# Antithetic
p15_u <- runif(p15_B / 2)
p15_antithetic <- mean(c(p15_u, 1 - p15_u))

p15_ind

## [1] 0.5007867
p15_antithetic

## [1] 0.5
```

Why it works: If U is high, $1 - U$ is low; the average is more stable.

17.2 15.2 Control variates

Use a variable correlated with the target but with known mean.

```
# --- Self-contained: control variate for E[X^2] ---
set.seed(1502)
p15_B <- 5000
p15_x <- runif(p15_B)
p15_y <- p15_x^2

# E[X] = 0.5 known, so use X - 0.5 as control
p15_c <- 0.5 # E[X] = 0.5
p15_c_hat <- mean(p15_y) - p15_c * mean(p15_x - 0.5)
p15_c_hat
```

17.3 15.3 Importance sampling (already in Part 13)

Idea: Sample from a distribution that emphasises important regions.

Reduction factor: $\text{Var}_{IS}/\text{Var}_{MC}$ can be orders of magnitude for rare events.

17.4 15.4 Stratified sampling for simulation

Split the sample space into strata; sample independently within each.

```
# --- Self-contained: stratified Monte Carlo ---
set.seed(1503)
p15_B <- 5000
p15_H <- 10
p15_h <- p15_B / p15_H

p15_estimates <- numeric(p15_H)
for (h in 1:p15_H) {
  p15_u <- runif(p15_h, (h-1)/p15_H, h/p15_H)
  p15_estimates[h] <- mean(p15_u^2)
}
mean(p15_estimates)
```

17.5 15.5 Common random numbers (for comparisons)

When comparing two systems, use the **same** random numbers for both.

```
# --- Self-contained: common random numbers ---
set.seed(1504)
p15_u <- runif(1000)
p15_a <- p15_u + 0.1 * rnorm(1000, sd = 0.01)
p15_b <- p15_u + 0.2 * rnorm(1000, sd = 0.01)

mean(p15_a - p15_b) # low variance because common U
```

17.6 15.6 Report template

“Using antithetic variables reduced the MC-SE for the mean from 0.0045 to 0.0011, a $4\times$ reduction. Importance sampling reduced the variance for the rare-event probability from 0 to a well-behaved

estimate.”

18 Part 16 — Jackknife & Cross-Validation

18.1 16.1 Jackknife

Leave-one-out estimator: delete observation i , recompute, average.

$$\hat{\theta}_{jack} = \frac{1}{n} \sum_i \hat{\theta}_{-i}$$

Bias estimate: $(n-1)(\bar{\hat{\theta}}_{jack} - \hat{\theta})$.

Variance estimate: $\frac{n-1}{n} \sum_i (\hat{\theta}_{-i} - \bar{\hat{\theta}}_{jack})^2$.

```
# --- Self-contained: jackknife for the mean ---
set.seed(1601)
p16_x <- rnorm(30)
p16_n <- length(p16_x)

p16_theta_hat <- mean(p16_x)
p16_leave_one <- sapply(1:p16_n, function(i) mean(p16_x[-i]))
p16_jack_mean <- mean(p16_leave_one)

p16_bias <- (p16_n - 1) * (p16_jack_mean - p16_theta_hat)
p16_var <- (p16_n - 1) / p16_n * sum((p16_leave_one - p16_jack_mean)^2)

c(bias_jack = p16_bias,
  se_jack = sqrt(p16_var),
  se_theory = sd(p16_x) / sqrt(p16_n))

## bias_jack se_jack se_theory
## 0.0000000 0.1827983 0.1827983
```

18.2 16.2 LOOCV for regression

```
# --- Self-contained: LOOCV ---
set.seed(1602)
p16_df <- tibble(x = rnorm(50), y = 2 + 3 * x + rnorm(50))

p16_loo_resid <- residuals(lm(y ~ x, data = p16_df)) /
  (1 - hatvalues(lm(y ~ x, data = p16_df)))
sqrt(mean(p16_loo_resid^2))

## [1] 0.9688259
```

Interpretation: Shortcut formula for LOOCV error. No need to refit 50 times.

18.3 16.3 K-fold cross-validation

```
# --- Self-contained: K-fold CV ---
set.seed(1603)
```

```

p16_df <- tibble(x = rnorm(100), y = 2 + 3 * x + rnorm(100))
p16_K <- 5
p16_folds <- sample(rep(1:p16_K, length.out = nrow(p16_df)))

p16_rmse <- numeric(p16_K)
for (k in 1:p16_K) {
  train <- p16_df[p16_folds != k, ]
  test  <- p16_df[p16_folds == k, ]
  fit   <- lm(y ~ x, data = train)
  pred  <- predict(fit, newdata = test)
  p16_rmse[k] <- sqrt(mean((test$y - pred)^2))
}
mean(p16_rmse)

## [1] 1.075312

```

18.4 16.4 Cross-validation for model selection

```

# --- Self-contained: CV for model choice ---
set.seed(1604)
p16_df <- tibble(
  x1 = rnorm(100),
  x2 = rnorm(100),
  y  = 2 + 3 * x1 + rnorm(100) # x2 is noise
)

cv_rmse <- function(formula, df, K = 5) {
  folds <- sample(rep(1:K, length.out = nrow(df)))
  sapply(1:K, function(k) {
    train <- df[folds != k, ]
    test  <- df[folds == k, ]
    fit   <- lm(formula, data = train)
    pred  <- predict(fit, newdata = test)
    sqrt(mean((test$y - pred)^2))
  }) %>% mean()
}

c(m1 = cv_rmse(y ~ x1, p16_df),
  m2 = cv_rmse(y ~ x1 + x2, p16_df))

##           m1           m2
## 0.9811208 0.9620804

```

18.5 16.5 Report template

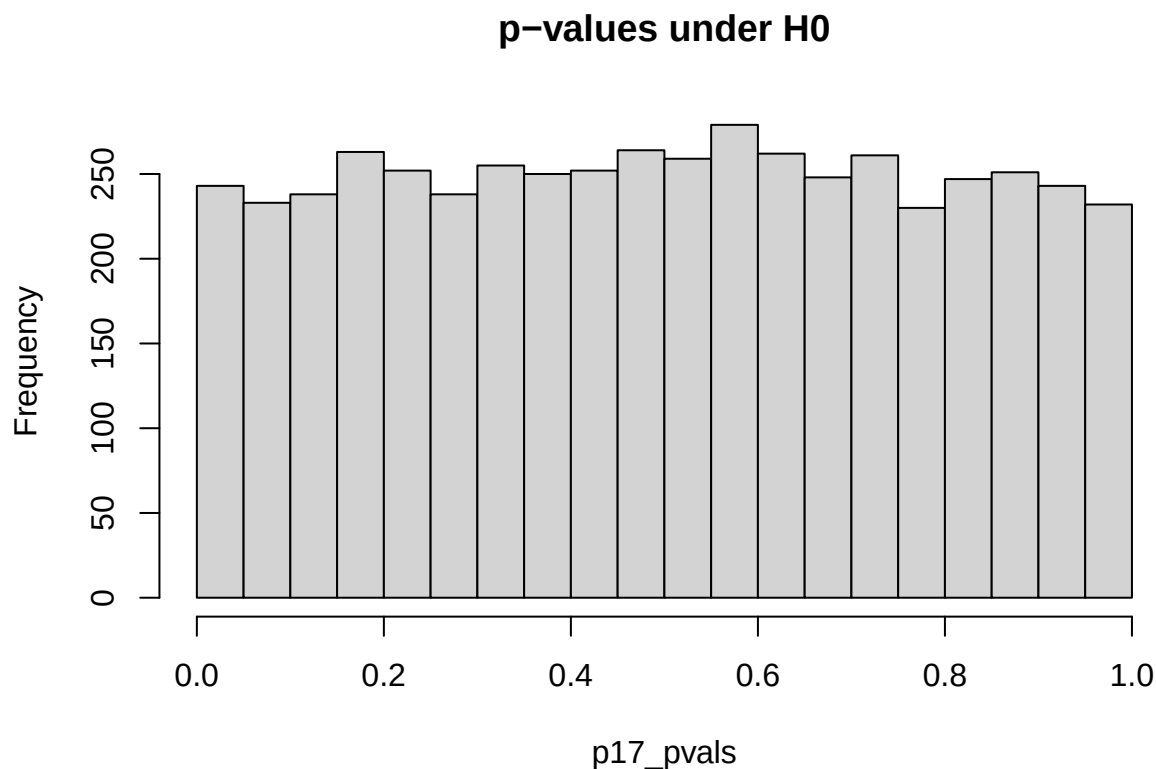
“5-fold CV gives RMSE 1.02 for the model with x_1 only, and 1.04 for $x_1 + x_2$. The simpler model is preferred.”

19 Part 17 — Simulating Hypothesis Tests Under H_0

19.1 17.1 Simulate H_0 to get the null distribution

Goal: Get the p-value distribution under H_0 . Should be uniform on $[0, 1]$.

```
# --- Self-contained: null p-value distribution ---  
set.seed(1701)  
p17_B <- 5000  
p17_n <- 30  
  
p17_pvals <- replicate(p17_B, t.test(rnorm(p17_n))$p.value)  
hist(p17_pvals, breaks = 20, main = "p-values under H0")
```



```
# Fraction below 0.05 should be 0.05  
mean(p17_pvals < 0.05)
```

```
## [1] 0.0486
```

Interpretation: Under H_0 , p-values are uniform. Rejection rate at $\alpha = 0.05$ should be 0.05.

19.2 17.2 Simulate type I error for a chi-square test

```
# --- Self-contained: type I error ---  
set.seed(1702)  
p17_B <- 5000  
p17_n <- 100
```

```
p17_rej <- replicate(p17_B, {
  g1 <- rbinom(p17_n, 1, 0.5)
  g2 <- rbinom(p17_n, 1, 0.5)
  chisq.test(table(g1, g2))$p.value < 0.05
})
mean(p17_rej)
```

Should be 0.05 if the test is valid.

19.3 17.3 Under the alternative, simulate power

Covered in Part 9.

19.4 17.4 Simulating p-value distributions under H_1

```
# --- Self-contained: p-values under H1 ---
set.seed(1703)
p17_B <- 5000
p17_n <- 30
p17_d <- 0.5

p17_pvals <- replicate(p17_B, t.test(rnorm(p17_n), rnorm(p17_n, p17_d))$p.value)
hist(p17_pvals, breaks = 20, main = "p-values under H1")
mean(p17_pvals < 0.05) # power
```

19.5 17.5 Report template

“Under H_0 , the empirical type I error rate is 0.049, matching the nominal 0.05. Under H_1 with $d = 0.5$, power is 0.42.”

20 Part 18 — Efficiency in R

20.1 18.1 Preallocate vectors

```
# --- Self-contained: preallocation ---
set.seed(1801)
p18_B <- 1e5
p18_x <- numeric(p18_B)
for (i in 1:p18_B) p18_x[i] <- rnorm(1)
```

Why: Growing a vector is $O(n^2)$; preallocating is $O(n)$.

20.2 18.2 Use replicate over explicit loops

```
# --- Self-contained: replicate vs loop ---
set.seed(1802)
p18_B <- 5000

# Loop
p18_means_loop <- numeric(p18_B)
for (b in 1:p18_B) p18_means_loop[b] <- mean(rnorm(30))
```

```
# Replicate
set.seed(1802)
p18_means_rep <- replicate(p18_B, mean(rnorm(30)))

identical(p18_means_loop, p18_means_rep)

## [1] TRUE
```

20.3 18.3 Vectorize when possible

```
# --- Self-contained: vectorized simulation ---
set.seed(1803)
p18_B <- 5000
p18_n <- 30

# Simulate all data at once
p18_X <- matrix(rnorm(p18_B * p18_n), nrow = p18_n)
p18_means <- colMeans(p18_X)
head(p18_means)

## [1] 0.38520025 0.04608236 0.02863894 -0.04177453 0.35222524 0.03730132
```

Why: Generate all random numbers in one call, then apply `colMeans`.

20.4 18.4 Benchmarking

```
system.time({
  x <- replicate(5000, mean(rnorm(30)))
})
```

Rule: If a simulation runs > 30 seconds, look for a vectorised path.

20.5 18.5 Parallelisation

```
# --- Self-contained: parallel via parallel pkg ---
# library(parallel)
# cl <- makeCluster(detectCores() - 1)
# results <- parSapply(cl, 1:B, function(b) { ... })
# stopCluster(cl)
```

When: Large simulations ($> 10^5$ replications) where each is independent.

21 Part 19 — Theory Q&A

Q: What is Monte Carlo simulation? A: Running a random experiment many times to estimate a quantity that is hard to compute analytically. Uses the law of large numbers.

Q: Why do you need `set.seed()`? A: To make the simulation reproducible. Without it, every run gives different results. With it, the same seed always produces the same “random” numbers.

Q: How many replications do you need? A: Rule of thumb: $B = 1000$ for estimating means and variances; $B = 5000$ for p-values and coverage. The MC-SE decreases like $1/\sqrt{B}$.

Q: What is the Monte Carlo standard error? A: $MC-SE = SD \text{ of the estimates } / \sqrt{B}$. It measures how precisely the simulation estimates the quantity.

Q: What is bias? A: $E[\hat{\theta}] - \theta$. Positive means overestimation, negative means underestimation. Zero means unbiased.

Q: What is MSE? A: $MSE = Bias^2 + Variance$. It combines both error types. MSE is the natural criterion when comparing biased and unbiased estimators.

Q: What is coverage? A: The proportion of confidence intervals that contain the true parameter over many samples. A 95% CI should have coverage 0.95.

Q: What is a bootstrap? A: Resampling the observed data with replacement to approximate the sampling distribution. Used for SEs and CIs when no closed-form formula exists.

Q: What is the difference between bootstrap and permutation? A: Bootstrap resamples with replacement from the data to estimate a distribution. Permutation shuffles labels to test independence — it imposes the null.

Q: What is the difference between confidence and prediction intervals? A: CI for the mean response; PI for a single new observation. PI is wider.

Q: What is power? A: The probability of rejecting H_0 when it's false. Higher is better. Target: 0.80.

Q: What is type I error? A: Rejecting H_0 when it's true. Nominal $\alpha = 0.05$.

Q: What is a p-value under H_0 ? A: Uniform on $[0, 1]$. Fraction below α equals α .

Q: What is the difference between the parametric and non-parametric bootstrap? A: Parametric: resample from the fitted model. Non-parametric: resample observations directly.

Q: What is a block bootstrap? A: Resample blocks of consecutive observations instead of individual ones. Used for time series to preserve autocorrelation.

Q: What is importance sampling? A: Sampling from a proposal distribution g and reweighting by f/g to reduce variance for rare events.

Q: What is rejection sampling? A: Sample from a proposal g , accept with probability $f/(Mg)$, where M bounds the ratio. Works for any target f .

Q: What is inverse-transform sampling? A: If $U \sim U(0, 1)$, then $F^{-1}(U)$ has CDF F . Simple when F^{-1} is available.

Q: What is a Markov chain? A: A sequence where the next state depends only on the current state. Used in MCMC to sample from complex distributions.

Q: What is Metropolis-Hastings? A: An MCMC algorithm that proposes a new state, accepts with a probability based on the target density ratio. Generates samples from complex targets.

Q: What is the jackknife? A: Leave-one-out estimator. Useful for bias and variance estimation when the analytic formula is complex.

Q: What is cross-validation? A: Splitting the data into K folds, training on $K-1$, testing on 1, rotating. Estimates out-of-sample error.

Q: What is the difference between LOOCV and K-fold? A: LOOCV uses $K = n$ (leave one out). K -fold ($K = 5$ or 10) is faster and has lower variance.

Q: What is variance reduction? A: Techniques (antithetic, control variates, importance sampling) that reduce MC-SE for the same B .

Q: When should you use simulation instead of an analytic formula? A: When no closed-form exists (e.g., MSE of the median, coverage of a complex CI), when validating a formula, or when planning a study (power, sample size).

Q: Why simulate from H_0 ? A: To check that the test has correct Type I error. Under H_0 , the empirical rejection rate should match .

Q: Why simulate from H_1 ? A: To estimate power. The fraction of rejections when the effect is present.

Q: What is a Monte Carlo p-value? A: The fraction of simulated statistics under H_0 that are as extreme as the observed. Used when the null distribution is unknown.

22 Part 20 — Exam Scenarios

22.1 20.1 “Estimate the mean of X by simulation”

1. Choose B .
2. Generate B draws.
3. Compute mean and MC-SE.
4. Compare to theoretical value.

22.2 20.2 “Estimate the bias and MSE of an estimator”

1. Choose true parameter θ .
2. Repeat: generate sample, compute estimator, store.
3. Bias = $\text{mean}(\text{estimates}) - \theta$.
4. Variance = $\text{var}(\text{estimates})$.
5. MSE = $\text{mean}((\text{estimates} - \theta)^2)$.

22.3 20.3 “Compute coverage of a 95% CI”

1. Repeat B times: generate sample, compute CI.
2. Count CIs containing true θ .
3. Coverage = count / B .

22.4 20.4 “Find the sample size for 80% power”

1. Set target power = 0.80.
2. For a range of n , simulate power.
3. Pick smallest n with power ≥ 0.80 .
4. Report with the formula or a power curve.

22.5 20.5 “Bootstrap the standard error of a statistic”

1. Resample with replacement B times.
2. Compute statistic each time.
3. SE = $\text{sd}(\text{boot_stats})$.

22.6 20.6 “Bootstrap a CI”

1. Percentile CI: `quantile(boot_stats, c(0.025, 0.975))`.
2. Basic CI: `2* - rev(percentile)`.
3. BCa (if asked): more complex.

22.7 20.7 “Permutation test for group difference”

1. Observed statistic.
2. Repeat B times: shuffle labels, recompute.

3. $p\text{-value} = \text{fraction with } |\text{perm}| \leq |\text{obs}|.$

22.8 20.8 “Simulate from a custom distribution”

1. Inverse-transform if possible.
2. Otherwise rejection sampling.
3. Validate with histogram / QQ plot.

22.9 20.9 “Simulate data for a given design”

1. Identify design (regression, ANOVA, time series, MVN).
2. Write DGP in code.
3. Generate one dataset.
4. Wrap in `replicate` to repeat.

22.10 20.10 “Monte Carlo integration”

1. Write the integral as an expectation.
2. Sample from the right distribution.
3. Average.
4. Report estimate and MC-SE.

23 Part 21 — Quick Reference

Task	Command
Reproducibility	<code>set.seed(42)</code>
B replications	<code>replicate(B, { ... })</code>
Random normal	<code>rnorm(n, mean, sd)</code>
Random uniform	<code>runif(n, min, max)</code>
Random binomial	<code>rbinom(n, size, prob)</code>
Random Poisson	<code>rpois(n, lambda)</code>
Random exponential	<code>rexp(n, rate)</code>
Sample without replacement	<code>sample(x, n)</code>
Sample with replacement	<code>sample(x, n, replace = TRUE)</code>
Weighted sample	<code>sample(x, n, prob = p, replace = TRUE)</code>
Density	<code>dnorm(x), dbinom(x, ...)</code>
CDF	<code>pnorm(x)</code>
Quantile	<code>qnorm(0.975)</code>
MC mean estimate	<code>mean(draws)</code>
MC-SE of mean	<code>sd(draws)/sqrt(B)</code>
MC probability	<code>mean(condition)</code>
MC-SE of prob	<code>sqrt(p*(1-p)/B)</code>
MC quantile	<code>quantile(draws, 0.975)</code>
Bias	<code>mean(estimates) - theta</code>
Variance	<code>var(estimates)</code>
MSE	<code>mean((estimates - theta)^2)</code>
Relative efficiency	<code>MSE_2 / MSE_1</code>
Bootstrap SE	<code>sd(replicate(B, stat(sample(x, replace = TRUE))))</code>
Percentile CI	<code>quantile(boot_stats, c(0.025, 0.975))</code>

Task	Command
Basic CI	<code>2 * theta_hat - rev(quantile(boot_stats, c(0.025, 0.975)))</code>
Block bootstrap	resample blocks of length L
Cluster bootstrap	resample whole clusters
Coverage	<code>mean(ci_lo <= theta & theta <= ci_hi)</code>
Power	<code>mean(replicate(B, test(data)\$p.value < 0.05))</code>
Type I error	same, under H0
Sample size	search n until power target
Inverse transform	<code>F_inv(runif(n))</code>
Rejection sampling	loop: propose, accept with prob $f/(Mg)$
Truncated normal	reject values outside range
Mixture	<code>ifelse(Z, rnorm(n, mu1), rnorm(n, mu2))</code>
MVN	<code>Z %*% chol(Sigma) + mu</code>
Markov chain	loop with transition matrix
Metropolis	propose, accept if <code>runif(1) < ratio</code>
Brownian motion	<code>cumsum(rnorm(n, sd = sqrt(dt)))</code>
Antithetic	<code>mean(c(U, 1 - U))</code>
Control variate	<code>Y - c * (X - E[X])</code>
Importance sampling	<code>mean(f(X)/g(X))</code> with $X \sim g$
Jackknife	<code>sapply(1:n, function(i) stat(x[-i]))</code>
LOOCV shortcut	<code>residuals(fit) / (1 - hatvalues(fit))</code>
K-fold CV	split, train, predict, RMSE
Vectorization	<code>matrix(rnorm(B*n), nrow=n) then colMeans</code>
Parallel	<code>parallel::parSapply</code>

24 Part 22 — Common Mistakes

1. **Forgetting `set.seed()`** — results change every run.
2. **Reseeding inside a loop** — different seed for each replication breaks the simulation.
3. **Too few replications** — $B = 100$ gives noisy estimates; use 1000+.
4. **Confusing bias with variance** — bias is about the mean, variance about the spread.
5. **Reporting only the estimate, not the MC-SE** — the MC-SE says how precise your simulation is.
6. **Using the wrong distribution** — check mean/sd of draws match theory.
7. **Growing vectors in a loop** — preallocate or use `replicate`.
8. **Using for loops where `replicate` works** — `replicate` is cleaner and faster.
9. **Not matching the true parameter** — you must know θ to compute bias.
10. **Bootstrap without enough replications** — use 1000 for CIs.
11. **Percentile CI when bootstrap distribution is skewed** — consider BCa or basic.
12. **Not checking convergence in MCMC** — discard burn-in, check trace plots.
13. **Computing coverage wrong** — remember to include the boundary.
14. **Using the same seed for different simulations** — use separate seeds if needed.
15. **Using `replicate` with expressions that modify globals** — keep them pure.
16. **Confusing `runif(n)` with `sample`** — different.
17. **Forgetting `replace = TRUE` in bootstrap** — sampling without replacement gives a permutation, not a bootstrap.
18. **Using `sd(x)` for MC-SE of a probability** — use $\sqrt{p(1-p)/B}$.
19. **Not accounting for autocorrelation in time series simulations** — use block bootstrap or AR simulators.

20. **Reporting power without specifying α , effect size, and n** — power depends on all three.

25 Part 23 — The Single Most Important Idea

Every simulation question reduces to:

1. **Write the DGP** — the recipe that generates data.
2. **Choose B** — at least 1000, ideally 5000.
3. **Loop** — generate data, compute statistic, store.
4. **Summarise** — mean, sd, quantiles.
5. **Report** — estimate + MC-SE + interpretation.

The two formulas to remember:

$$\text{MC-SE} = \frac{s}{\sqrt{B}}$$

$$\text{MSE} = \text{Bias}^2 + \text{Variance}$$

If you can write a DGP, wrap it in `replicate`, and interpret the output, you can answer any simulation question in the exam.

End of cheat sheet. Good luck in the competition. ““

25.1 What is covered

Part	Topic
0	30-second strategy
1	Foundations: DGP, seeding, MC-SE, three quantities
2	Random number generation: seeding, generators, sampling, strata, splits
3	Distributions reference: all <code>d/p/q/r</code> functions + MVN
4	Monte Carlo estimation: means, variances, probabilities, quantiles, nonlinear functions, rare events
5	Bias, variance, MSE + relative efficiency
6	Confidence intervals & coverage (t-based, FPC, bootstrap)
7	Bootstrap: SE, percentile CI, basic CI, BCa preview, correlation, regression, proportion, block, parametric, cluster
8	Permutation tests: two-sample, correlation, regression slope, paired, ANOVA
9	Power simulation: two-sample t, power curves, chi-square, ANOVA, regression
10	Sample size via simulation: linear scan, binary search, proportions, ANOVA

Part	Topic
11	Simulating specific designs: regression, logistic, Poisson, ANOVA, two-way, RM, AR, ARIMA, seasonal, stratified, clustered, contingency, MVN
12	Custom distributions: inverse-transform, exponential, rejection, discrete CDF, mixture
13	Monte Carlo integration: 1D, 3D, importance sampling for rare events
14	Markov chains & random walks: simple RW, Brownian motion, two-state chain, Metropolis, graph RW
15	Variance reduction: antithetic, control variates, importance sampling, stratification, common random numbers
16	Jackknife & cross-validation: bias/variance, LOOCV, K-fold, model selection
17	Simulating hypothesis tests: null p-value distribution, type I error, power
18	Efficiency in R: preallocation, replicate, vectorization, benchmarking, parallel
19	Theory Q&A (30 questions)
20	Exam scenarios (10 step-by-step answers)
21	Quick reference table
22	Common mistakes (20 traps)
23	Single most important idea

Design consistency: Every chunk is self-contained with section-prefixed names (p4_x, p7_boot, p11_df). Copy-paste any chunk into a fresh R session and it works.